Classical Machine Learning Notes

Author
K. Yu
2026

Contents

1 Linear Regression **6**
1.1 Overview . . . 6
1.2 Model Formulation . . . 6
1.3 Least-Squares Objective . . . 7
1.4 Optimisation by Gradient Descent . . . 7
1.5 Stochastic Gradient Descent . . . 8
1.6 Closed-form solution: The Normal Equation . . . 8
1.7 Probabilistic Interpretation . . . 9
1.8 Geometric and Practical Intuition . . . 9
1.9 Locally Weighted Linear Regression . . . 9
1.10 Strengths and Limitations . . . 10

2 Logistic Regression **11**
2.1 Overview . . . 11
2.2 Model Formulation . . . 11
2.3 Decision Rule and Decision Boundary . . . 12
2.4 Why not use Linear Regression for Classification? . . . 12
2.5 Probabilistic Model . . . 12
2.6 Likelihood and Loss Function . . . 12
2.7 Gradient of the Loss . . . 13
2.8 Optimisation . . . 13
2.9 Convexity . . . 14
2.10 Log-Odds Interpretation . . . 14
2.11 Regularisation . . . 15
2.12 Multi-Class Classification . . . 15
2.13 Relation to the Perceptron . . . 15
2.14 Geometric Intuition . . . 16
2.15 Strengths and Limitations . . . 16

3 Generalised Linear Models (GLM) **17**
3.1 Overview . . . 17
3.2 The Exponential Family . . . 17
3.3 Examples of Exponential Family Distributions . . . 18

3.4 Constructing a GLM . . . 18
3.5 Canonical Response Function and Link Function . . . 19
3.6 Linear Regression as a GLM . . . 19
3.7 Logistic Regression as a GLM . . . 20
3.8 Why GLMs matter? . . . 21
3.9 General Recipe for constructing a GLM . . . 21
3.10 Geometric and Statistical View . . . 21

4 Generative Models **22**
4.1 Overview . . . 22
4.2 Gaussian Discriminant Analysis (GDA) . . . 22
4.3 Maximum Likelihood Estimation in GDA . . . 24
4.4 Prediction in GDA . . . 25
4.5 Relationship between GDA and Logistic Regression . . . 25
4.6 Naive Bayes . . . 26
4.7 Naive Bayes Assumption . . . 26
4.8 Bernoulli Naive Bayes . . . 27
4.9 Laplacing Smoothing . . . 27
4.10 Multinomial Event Model for Text . . . 28
4.11 GDA vs Naive Bayes . . . 29
4.12 Advantages and Limitations . . . 30

5 k-Nearest Neighbours **31**
5.1 Overview . . . 31
5.2 Problem Setup . . . 31
5.3 Distance Metrics . . . 31
5.4 The kNN Rule . . . 32
5.5 kNN for Classification . . . 32
5.6 kNN for Regression . . . 32
5.7 Algorithm . . . 33
5.8 Geometric Intuition . . . 33
5.9 Why kNN is Non-Parametric . . . 34
5.10 Choice of k . . . 34
5.11 Importance of Feature Scaling . . . 35
5.12 Curse of Dimensionality . . . 35
5.13 Computational Complexity . . . 36
5.14 Weighted kNN and Local Smoothing . . . 36
5.15 Statistical Interpretation . . . 36
5.16 Strengths and Limitations . . . 37

6 Decision Trees **38**
6.1 Overview . . . 38
6.2 Model Formulation . . . 38
6.3 Tree Representation . . . 38
6.4 Prediction at Leaves . . . 38
6.5 Fitting a Tree . . . 39

6.6 Greedy Splitting . . . 39
6.7 Gain and Cost . . . 40
6.8 Split Criteria for Regression and Classification . . . 40
6.9 Algorithm . . . 41
6.10 Regularisation and Pruning . . . 41
6.11 Strengths and Weaknesses . . . 42

7 Ensemble Methods **43**
7.1 Overview . . . 43
7.2 Bagging . . . 43
7.3 Random Forests . . . 44
7.4 Boosting . . . 44
7.5 Forward Stagewise Additive Modelling . . . 45
7.6 Gradient Boosting . . . 45
7.7 Gradient-Boosted Trees . . . 46
7.8 XGBoost . . . 46
7.9 Comparison . . . 47

8 Kernel Methods **48**
8.1 Overview . . . 48
8.2 Feature Maps . . . 48
8.3 Least Mean Squares in Feature Space . . . 49
8.4 Why Explicit Feature Maps can be expensive? . . . 49
8.5 Kernel Trick via Representer Form . . . 49
8.6 Kernalised LMS . . . 50
8.7 Polynomial Kernels . . . 51
8.8 Gaussian Kernel . . . 51
8.9 Kernel Matrix and Valid Kernels . . . 51
8.10 Mercer Condition . . . 52

9 Support Vector Machines **53**
9.1 Binary Linear Classification Setup . . . 53
9.2 Function Margin . . . 53
9.3 Geometric Margin . . . 53
9.4 Geometric meaning of Margin . . . 54
9.5 Hard-Margin SVM . . . 55
9.6 Lagrangian and Dual Problem . . . 55
9.7 Support Vectors . . . 56
9.8 Why Kernelisation works for SVM . . . 56
9.9 Soft-Margin SVM . . . 57
9.10 Dual of Soft-Margin SVM . . . 57
9.11 Coordinate Ascent and SMO . . . 58
9.12 Geometric and Statistical Interpretation of SVM . . . 59
9.13 Comparison with Logistic Regression . . . 59

10 Generalisation **61**
10.1 Overview . . . 61
10.1.1 Main questions . . . 61
10.2 Training Error vs Test Error . . . 61
10.3 Underfitting and Overfitting . . . 62
10.4 Bias and Variance: Informal View . . . 62
10.5 Bias-Variance Decomposition for Regression . . . 62
10.6 Bias-Variance Tradeoff . . . 63
10.7 Double Descent . . . 64

11 Regularisation **66**
11.1 Overview . . . 66
11.2 ℓ_2 Regularisation . . . 66
11.3 ℓ_1 Regularisation and Sparsity . . . 67
11.4 Regularisation beyond Linear Models . . . 67
11.5 Implicit Regularisation . . . 67
11.6 Model Selection and Cross Validation . . . 68
11.7 Hold-Out Cross Validation . . . 69
11.8 k-Fold Cross Validation . . . 69
11.9 Bayesian View and MAP Estimation . . . 70
11.10 MAP and Regularisation . . . 70
11.11 Big Picture . . . 71

12 K-Means Clustering **72**
12.1 Overview . . . 72
12.2 Problem Formulation . . . 72
12.3 The K-Means Algorithm . . . 73
12.4 Distortion Function . . . 73
12.5 K-means as Coordinate Descent . . . 74
12.6 Initialisation and Multiple Restarts . . . 75
12.7 Geometric Interpretation . . . 75
12.8 Choosing K . . . 76
12.9 Empty Clusters . . . 76
12.10 Strengths and Limitations . . . 77
12.11 Importance of Feature Scaling . . . 77

13 Density-based Clustering **78**
13.1 DBSCAN: core definitions . . . 78
13.2 DBSCAN Algorithm . . . 79
13.3 Advantages and Limitations of DBSCAN . . . 80
13.4 Choose ε and MinPts . . . 81
13.5 Complexity . . . 81

14 Hierarchical Clustering **82**
14.1 Two paradigms . . . 82
14.2 Agglomerative clustering framework . . . 82
14.3 Linkage criteria . . . 83
14.4 Agglomerative clustering algorithm . . . 84
14.5 Properties of different linkage rules . . . 84
14.6 Output: obtaining a flat clustering . . . 84
14.7 Complexity . . . 85
14.8 Advantages and Limitations . . . 85

15 Principal Components Analysis **86**
15.1 Overview . . . 86
15.2 Data Centering . . . 86
15.3 First Principal Component: Variance Maximisation View . . . 86
15.4 Eigenvector Characterisation . . . 87
15.5 Subsequent Principal Components . . . 88
15.6 Projection onto the Principal Subspace . . . 88
15.7 Reconstruction Error View . . . 89
15.8 Explained Variance . . . 89
15.9 Matrix Formulation . . . 90
15.10 Geometric Interpretation . . . 91
15.11 Whitening . . . 91
15.12 Practical Issues . . . 91
15.13 Strengths and Limitations . . . 92

Supervised Learning

1 Linear Regression

1.1 Overview

Linear regression is a supervised learning method for predicting a **continuous-valued output** from an input vector. The hypothesis is taken to be a linear function of the input,

$$h_{\boldsymbol{\theta}}(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d,$$

or equivalently, after defining $x_0 = 1$,

$$h_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta}^\top \mathbf{x}.$$

This is the basic model used to fit a line, plane, or hyperplane to data.

When to use

Use linear regression when:

- the target variable is continuous;
- the relationship is approximately linear in the chosen features;
- interpretability matters;
- we want a strong baseline before trying more complex models.

1.2 Model Formulation

Definition 1.2.1 (Linear regression hypothesis)**.** Let $\mathbf{x} \in \mathbb{R}^{d+1}$ with $x_0 = 1$. A linear regression model is defined by

$$h_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta}^\top \mathbf{x},$$

where $\boldsymbol{\theta} \in \mathbb{R}^{d+1}$ is the parameter vector.

The training set of n training example is

$$\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n.$$

The goal is to choose $\boldsymbol{\theta}$ so that predictions $h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})$ are close to targets $y^{(i)}$.

1.3 Least-Squares Objective

Definition 1.3.1 (Squared error cost)**.** The standard cost function for linear regression is

$$J(\boldsymbol{\theta}) = \frac{1}{2}\sum_{i=1}^{n}\left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)}\right)^2.$$

The factor 1/2 is included only for convenience, since it simplifies derivatives.

Interpretation

Minimising $J(\boldsymbol{\theta})$ means finding the linear function that best fits the training data in the sense of minimising the total squared prediction error.

1.4 Optimisation by Gradient Descent

To minimise $J(\boldsymbol{\theta})$, one approach is gradient descent:

$$\theta_j \leftarrow \theta_j - \alpha\frac{\partial}{\partial\theta_j}J(\boldsymbol{\theta}),$$

for $j = 0, 1, \ldots, d$, where $\alpha > 1$ is the learning rate.

For one example $(\mathbf{x}, y)$,

$$\frac{\partial}{\partial\theta_j}J(\boldsymbol{\theta}) = (h_{\boldsymbol{\theta}}(\mathbf{x}) - y)x_j.$$

Hence the update becomes

$$\theta_j \leftarrow \theta_j - \alpha(h_{\boldsymbol{\theta}}(\mathbf{x}) - y)x_j.$$

This is the **LMS update rule** (least mean squares), also called the Widrow-Hoff rule.

Algorithm 1 Batch Gradient Descent

Require: training set $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{n}$, learning rate α
  Initialise $\boldsymbol{\theta}$
  while $\boldsymbol{\theta}$ not converged **do**
    for $j \in \{0, 1, \ldots, d\}$ **do**
      $\theta_j \leftarrow \theta_j - \alpha\sum_{i=1}^{n}(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)})x_j^{(i)}$
    end for
  end while
  return $\boldsymbol{\theta}$

This is called **batch gradient descent** because each update uses the entire training set. For linear regression this objective has a single global optimum, since J is a convex quadratic function.

Proposition 1.4.1. For linear regression with squared loss, $J(\boldsymbol{\theta})$ is convex in $\boldsymbol{\theta}$.

Any local minimum is a global minimum. Therefore, with a suitable learning rate, gradient descent converges to the global minimiser.

1.5 Stochastic Gradient Descent

Instead of updating using all examples at once, we may update after each single example

$$\theta_j \leftarrow \theta_j - \alpha(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)})x_j^{(i)}.$$

Algorithm 2 Stochastic Gradient Descent

Require: training set $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, learning rate α
  Initialise $\boldsymbol{\theta}$
  while $\boldsymbol{\theta}$ not converged **do**
    for $i \in \{1, \ldots, n\}$ **do**
      for $j \in \{0, 1, \ldots, d\}$ **do**
        $\theta_j \leftarrow \theta_j - \alpha(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)})x_j^{(i)}$
      end for
    end for
  end while
  return $\boldsymbol{\theta}$

Batch gradient descent computes a more stable direction but requires a full pass over the dataset per step. Stochastic gradient descent makes progress immediately on each example and is often much faster on large datasets, though the iterates may oscillate around the optimum rather than converge exactly.

1.6 Closed-form solution: The Normal Equation

Linear regression also admits a closed-form solution. Let

$$X = \begin{bmatrix} - & (\mathbf{x}^{(1)})^\top & - \\ - & (\mathbf{x}^{(2)})^\top & - \\ & \vdots & \\ - & (\mathbf{x}^{(n)})^\top & - \end{bmatrix} \in \mathbb{R}^{n \times (d+1)}, \quad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix}.$$

Then the objective can be written as

$$J(\boldsymbol{\theta}) = \frac{1}{2}(X\boldsymbol{\theta} - \mathbf{y})^\top (X\boldsymbol{\theta} - \mathbf{y}).$$

Differentiating and setting the gradient to zero gives

$$X^\top X\boldsymbol{\theta} = X^\top \mathbf{y}.$$

Theorem 1.6.1 (Normal equation solution)**.** If $X^\top X$ is invertible, then the unique minimiser of the least-squares objective is

$$\boldsymbol{\theta}^* = (X^\top X)^{-1} X^\top \mathbf{y}.$$

If $X^\top X$ is not invertible, the features may be linearly dependent or there may be insufficient independent examples. In practice, gradient-based optimisation is often preferred for very large datasets.

1.7 Probabilistic Interpretation

Linear regression is not only an optimisation procedure; it also has probabilistic justification. Assume

$$y^{(i)} = \boldsymbol{\theta}^\top \mathbf{x}^{(i)} + \varepsilon^{(i)},$$

where the noise terms are i.i.d. Gaussian, $\varepsilon^{(i)} \sim \mathcal{N}(0, \sigma^2)$. Then

$$y^{(i)} | \mathbf{x}^{(i)}; \boldsymbol{\theta} \sim \mathcal{N}(\boldsymbol{\theta}^\top \mathbf{x}^{(i)}, \sigma^2).$$

Under this model, the likelihood is

$$L(\boldsymbol{\theta}) = \prod_{i=1}^{n} p(y^{(i)} | \mathbf{x}^{(i)}; \boldsymbol{\theta}).$$

Taking logs shows that maximising the log-likelihood is equivalent to minimising

$$\frac{1}{2} \sum_{i=1}^{n} (\boldsymbol{\theta}^\top \mathbf{x}^{(i)} - y^{(i)})^2.$$

So least-squares regression is precisely **maximum likelihood estimation under Gaussian noise**.

Proposition 1.7.1. Under the assumptions,

$$y^{(i)} = \boldsymbol{\theta}^\top \mathbf{x}^{(i)} + \varepsilon^{(i)}, \quad \varepsilon^{(i)} \overset{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2),$$

the MLE of $\boldsymbol{\theta}$ coincides with the least-squares estimator.

1.8 Geometric and Practical Intuition

Geometric view

Linear regression chooses the parameter vector $\boldsymbol{\theta}$ that makes $X\boldsymbol{\theta}$ as close as possible to $\mathbf{y}$ in Euclidean distance.

Practical view

The model is simple, interpretable, and fast. But its performance depends heavily on feature design:

- if the true relationship is close to linear, linear regression can work very well;
- if the relationship is nonlinear, performance may degrade unless we enrich the features.

1.9 Locally Weighted Linear Regression

A non-parametric extension of linear regression, instead of minimising

$$\sum_{i=1}^{n} (\boldsymbol{\theta}^\top \mathbf{x}^{(i)} - y^{(i)})^2,$$

we minimise

$$\sum_{i=1}^{n} w^{(i)}(\boldsymbol{\theta}^\top \mathbf{x}^{(i)} - y^{(i)})^2,$$

where the weights depend on the query point $\mathbf{x}$. A standard choice is

$$w^{(i)} = \exp\left(-\frac{\|\mathbf{x}^{(i)} - \mathbf{x}\|^2}{2\tau^2}\right).$$

Thus, nearby points receive larger weight, and distant points receive smaller weight. The bandwidth parameter τ controls how local the fit is.

Local weighted linear regression is useful when the relationship is locally linear but globally nonlinear.

1.10 Strengths and Limitations

Strengths

- simple and interpretable;
- efficient to train;
- convex optimisation;
- probabilistically well-motivated;
- strong baseline for regression tasks.

Limitations

- only models linear dependence in the chosen features;
- sensitive to poor feature engineering;
- can underfit nonlinear patterns;
- normal equation can be numerically problematic if $X^\top X$ is singular or ill-conditioned.

2 Logistic Regression

2.1 Overview

Logistic regression is a supervised learning method for **binary classification**. Unlike linear regression, whose output is a real number, logistic regression models the probability that an example belongs to class 1. Given an input vector $\mathbf{x} \in \mathbb{R}^{d+1}$ with $x_0 = 1$, logistic regression predicts

$$P(y = 1|\mathbf{x}; \boldsymbol{\theta})$$

through a nonlinear transformation of a linear score.

We first compute a linear score

$$z = \boldsymbol{\theta}^\top \mathbf{x} = \theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d,$$

and then map it into $(0, 1)$ using the sigmoid function.

When to use

Use logistic regression when:

- the target is binary;
- interpretability matters;
- we want a fast and strong classification baseline;
- we need probabilistic outputs rather than only class labels.

2.2 Model Formulation

Definition 2.2.1 (Sigmoid function)**.** The sigmoid, or logistic, function $\sigma : \mathbb{R} \to (0, 1)$ is

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

It satisfies $\sigma(-z) = 1 - \sigma(z)$, and $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

Definition 2.2.2 (Logistic regression hypothesis)**.** For binary labels $y \in \{0, 1\}$, logistic regression defines

$$h(\mathbf{x}) = P(y = 1|\mathbf{x}) = \sigma(\boldsymbol{\theta}^\top \mathbf{x}).$$

Equivalently,

$$P(y = 0|\mathbf{x}) = 1 - h(\mathbf{x}).$$

Interpretation

The quantity $\boldsymbol{\theta}^\top \mathbf{x}$ is the **logit** or linear score, and the sigmoid converts it into a probability.

2.3 Decision Rule and Decision Boundary

The predicted class is usually

$$\hat{y} = \begin{cases} 1 & \text{if } h(\mathbf{x}) > \frac{1}{2}, \\ 0 & \text{if } h(\mathbf{x}) < \frac{1}{2}. \end{cases}$$

Since $\sigma(z) \geq \frac{1}{2}$ if and only if $z \geq 0$, this is equivalent to

$$\hat{y} = \begin{cases} 1 & \text{if } \boldsymbol{\theta}^\top \mathbf{x} \geq 0, \\ 0 & \text{if } \boldsymbol{\theta}^\top \mathbf{x} < 0. \end{cases}$$

Proposition 2.3.1. The decision boundary of logistic regression is

$$\boldsymbol{\theta}^\top \mathbf{x} = 0.$$

Thus, although the output probability is nonlinear in the score, the classifier is still **linear in feature space**.

2.4 Why not use Linear Regression for Classification?

A natural question is: why not fit a linear model and threshold the output?

Main problems

1. **Outputs are unbounded**: linear regression can predict values below 0 or above 1, which are not valid probabilities.
2. **Loss mismatch**: squared error is not the natural loss for Bernoulli labels.
3. **Probabilistic interpretation**: logistic regression corresponds naturally to a Bernoulli model and maximum likelihood.

So logistic regression is the standard probabilistic linear classifier.

2.5 Probabilistic Model

Definition 2.5.1 (Bernoulli model)**.** Assume $y|\mathbf{x} \sim \text{Bernoulli}(h(\mathbf{x})$, where $h(\mathbf{x}) = \sigma(\boldsymbol{\theta}^\top \mathbf{x})$. Then

$$P(y|\mathbf{x}) = h(\mathbf{x})^y (1 - h(\mathbf{x}))^{1-y}.$$

This compact form works because if $y = 1$, it becomes $h(\mathbf{x})$; if $y = 0$, it becomes $1 - h(\mathbf{x})$.

2.6 Likelihood and Loss Function

Given training data $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, where $y^{(i)} \in \{0, 1\}$, the likelihood is

$$L(\boldsymbol{\theta}) = \prod_{i=1}^n P(y^{(i)}|\mathbf{x}^{(i)}) = \prod_{i=1}^n h(\mathbf{x}^{(i)})^{y^{(i)}} (1 - h(\mathbf{x}^{(i)}))^{1-y^{(i)}}.$$

Taking logs gives the log-likelihood:

$$\ell(\boldsymbol{\theta}) = \sum_{i=1}^{n} \Big[y^{(i)} \log h(\mathbf{x}^{(i)}) + (1 - y^{(i)}) \log(1 - h(\mathbf{x}^{(i)})) \Big].$$

We maximise ℓ, or equivalently, minimise the negative log-likelihood.

Definition 2.6.1 (Binary cross-entropy loss)**.** The empirical loss is

$$J(\boldsymbol{\theta}) = -\sum_{i=1}^{n} \Big[y^{(i)} \log h(\mathbf{x}^{(i)}) + (1 - y^{(i)}) \log(1 - h(\mathbf{x}^{(i)})) \Big].$$

This is also called **log loss**, **cross-entropy loss**, **negative log-likelihood**.

Takeaway

$$\text{Logistic Regression} = \text{linear score} + \text{sigmoid} + \text{Bernoulli maximum likelihood}$$

2.7 Gradient of the Loss

Let $z^{(i)} = \boldsymbol{\theta}^\top \mathbf{x}^{(i)}$, $h^{(i)} = \sigma(z^{(i)})$. Then for one example, the loss is

$$\ell^{(i)} = -\Big[y^{(i)} \log h^{(i)} + (1 - y^{(i)}) \log(1 - h^{(i)}) \Big].$$

Using the chain rule and $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, we obtain

$$\frac{\partial J}{\partial \boldsymbol{\theta}} = \sum_{i=1}^{n} (h^{(i)} - y^{(i)}) \mathbf{x}^{(i)}.$$

Proposition 2.7.1. For logistic regression, the gradient has the same high-level form as in linear regression:

$$\text{prediction} - \text{target}.$$

This is a useful mental model:

- compute the prediction,
- compare it to the true label,
- move parameters in the direction that reduces the error.

2.8 Optimisation

Unlike ordinary least squares, logistic regression has **no closed-form normal equation solution**. So we optimise iteratively.

- Batch GD is stable but expensive for large n.
- SGD updates faster and is more scalable.

Algorithm 3 Batch Gradient Descent for Logistic Regression

Require: training set $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, learning rate α
  Initialise $\boldsymbol{\theta}$
  while $\boldsymbol{\theta}$ not converged **do**
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \sum_{i=1}^n (\sigma(\boldsymbol{\theta}^\top \mathbf{x}^{(i)}) - y^{(i)})\mathbf{x}^{(i)}$
  end while
  return $\boldsymbol{\theta}$

Algorithm 4 Stochastic Gradient Descent for Logistic Regression

Require: training set $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, learning rate α
  Initialise $\boldsymbol{\theta}$
  while $\boldsymbol{\theta}$ not converged **do**
    for $i \in \{1, \ldots, n\}$ **do**
      $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha(\sigma(\boldsymbol{\theta}^\top \mathbf{x}^{(i)}) - y^{(i)})\mathbf{x}^{(i)}$
    end for
  end while
  return $\boldsymbol{\theta}$

- In practice, mini-batch gradient descent is often used.

2.9 Convexity

Theorem 2.9.1. The logistic regression negative log-likelihood is convex in $\boldsymbol{\theta}$.

Any local minimum is a global minimum. This is one of the major advantages of logistic regression:

- the model is nonlinear in output,
- but the optimisation problem is still well-behaved.

2.10 Log-Odds Interpretation

One elegant interpretation is through the **odds**:

$$\frac{P(y=1|\mathbf{x})}{P(y=0|\mathbf{x})} = \frac{h(\mathbf{x})}{1-h(\mathbf{x})}.$$

Taking logarithms,

$$\log \frac{h(\mathbf{x})}{1-h(\mathbf{x})} = \boldsymbol{\theta}^\top \mathbf{x}.$$

Definition 2.10.1 (Log-odds/logit)**.** The quantity

$$\log \frac{P(y=1|\mathbf{x})}{P(y=0|\mathbf{x})}$$

is called the **log-odds** or **logit**.

Logistic regression says that the **log-odds are linear in the features**.

2.11 Regularisation

To reduce overfitting, we often add a penalty on the weights.

L_2-regularisation objective

$$J(\boldsymbol{\theta}) + \frac{\lambda}{2}\|\mathbf{w}\|_2^2$$

L_1-regularisation objective

$$J(\boldsymbol{\theta}) + \lambda\|\mathbf{w}\|_1$$

Practical difference

- L_2: shrinks weights smoothly.
- L_1: encourages sparsity and feature selection.

2.12 Multi-Class Classification

There are two common approaches.

One-vs-Rest

Train K binary classifiers, one for each class:

- classifier k: class k vs not class k.

St test time, choose the class with highest score or highest predicted probability.

Softmax Regression

For K classes, define scores

$$s_k = \boldsymbol{\theta}_k^\top \mathbf{x}.$$

Then

$$P(y = k|\mathbf{x}) = \text{softmax}(s_k) = \frac{e^{s_k}}{\sum_{j=1}^{K} e^{s_j}}.$$

This is the multiclass generalisation of logistic regression.

2.13 Relation to the Perceptron

Similarity

Both are linear classifiers of the form

$$\boldsymbol{\theta}^\top \mathbf{x}.$$

Difference

- **Perceptron**: predicts hard labels and uses a mistaken-driven update.
- **Logistic regression**: predicts probabilities and optimises a smooth likelihood-based loss.

Why logistic regression is preferable?

- probabilistic interpretation,
- differentiable loss,
- better optimisation behaviour,
- more useful in practice.

2.14 Geometric Intuition

Although logistic regression outputs probabilities, geometrically it still builds a hyperplane:

$$\boldsymbol{\theta}^\top \mathbf{x} = 0.$$

- Points far on one side have probability close to 1,
- points far on the other side have probability close to 0,
- points near the boundary are uncertain.

The vector $\boldsymbol{\theta}$ is normal to the boundary. Its magnitude affects how sharply the probability transitions across the boundary.

2.15 Strengths and Limitations

Strengths

- Simple and fast;
- interpretable;
- probabilistic outputs;
- convex optimisation;
- strong baseline for binary classification.

Limitations

- Linear decision boundary in feature space;
- may underfit complex nonlinear data;
- sensitive to poor feature engineering;
- performance can degrade under severe class imbalance unless adjusted.

3 Generalised Linear Models (GLM)

3.1 Overview

Both of linear regression and logistic regression are special cases of a broader family called **Generalised Linear Models** (GLMs). The purpose of GLMs is to provide a unified recipe for building models of the form

$$p(y|\mathbf{x};\boldsymbol{\theta}),$$

where the conditional distribution of y belongs to a suitable probabilisitic family, and where the model remains tractable to learn.

Main idea

A GLM combines three ingredients:

1. a distribution for $y|\mathbf{x}$ from the **exponential family**;
2. a notion of prediction based on the **conditional mean**;
3. a **linear relation** between the natural parameter and the input features.

This explains why linear regression and logistic regression have similar structural forms despite solving different problems.

3.2 The Exponential Family

Definition 3.2.1 (Exponential family)**.** A distribution is in the exponential family if it can be written as

$$p(y;\boldsymbol{\eta}) = b(y)\exp(\boldsymbol{\eta}^\top T(y) - a(\boldsymbol{\eta})),$$

where:

- $\boldsymbol{\eta}$ is the **natural parameter** (or canonical parameter),
- $T(y)$ is the **sufficient statistic**,
- $a(\boldsymbol{\eta})$ is the **log-partition function**,
- $b(y)$ depends only on y.

Interpretation

This form is important because many common distributions can be rewritten in it. Once this is done, a unified modelling framework becomes possible.

Why $a(\boldsymbol{\eta})$ matters?

The term $e^{-a(\boldsymbol{\eta})}$ acts as the normalisation term ensuring the distribution sums or integrates to 1.

3.3 Examples of Exponential Family Distributions

Bernoulli distribution

For $y \in \{0, 1\}$, the Bernoulli distribution with parameter ϕ is

$$p(y; \phi) = \phi^y (1 - \phi)^{1-y}.$$

Rewriting it,

$$p(y; \phi) = \exp\left[y \log \frac{\phi}{1 - \phi} + \log(1 - \phi)\right].$$

Thus, it can be written in exponential family form with

$$\eta = \log \frac{\phi}{1 - \phi}, \quad T(y) = y, \quad a(\eta) = \log(1 + e^\eta), \quad b(y) = 1.$$

So Bernoulli belongs to the exponential family. Solving for ϕ in terms of η,

$$\phi = \frac{1}{1 + e^{-\eta}}.$$

This is exactly the **sigmoid function**, which is why logistic regression naturally arises as a GLM for Bernoulli data.

Gaussian distribution

For a Gaussian with mean μ and variance fixed to 1,

$$p(y; \mu) = \frac{1}{\sqrt{2\pi}} \exp\left[-\frac{(y - \mu)^2}{2}\right].$$

This can be rewritten as

$$p(y; \mu) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{y^2}{2}\right) \exp\left(\mu y - \frac{\mu^2}{2}\right).$$

Hence the Gaussian is also in the exponential family, with

$$\eta = \mu, \quad T(y) = y, \quad a(\eta) = \frac{\eta^2}{2}, \quad b(y) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{y^2}{2}\right).$$

3.4 Constructing a GLM

Suppose we want to predict y from input $\mathbf{x}$.

Assumption 1

The conditional distribution of y given $\mathbf{x}$ belongs to the exponential family:

$$y|\mathbf{x} \sim \text{ExponentialFamily}(\eta).$$

Assumption 2

Our prediction is the conditional expectation of the sufficient statistic, usually just the conditional mean:

$$h_{\boldsymbol{\theta}}(\mathbf{x}) = E(y|\mathbf{x};\boldsymbol{\theta}),$$

since in most examples $T(y) = y$.

Assumption 3

The natural parameter depends linearly on the input:

$$\eta = \boldsymbol{\theta}^\top \mathbf{x}.$$

If η is vector-valued, then each component is linear in $\mathbf{x}$.

3.5 Canonical Response Function and Link Function

Definition 3.5.1 (Canonical response function)**.** The function

$$g(\eta) = E[T(y);\eta]$$

is called the **canonical response function**.

In the common scalar case with $T(y) = y$, this becomes

$$g(\eta) = E(y|\eta).$$

Definition 3.5.2 (Canonical link function)**.** The inverse function g^{-1} is called the **canonical link function**.

Interpretation

The GLM recipe can be summarised as:

1. compute the linear predictor $\eta = \boldsymbol{\theta}^\top \mathbf{x}$;
2. apply the response function g to obtain the mean;
3. use that mean to define the model prediction.

Thus,

$$h_{\boldsymbol{\theta}}(\mathbf{x}) = g(\boldsymbol{\theta}^\top \mathbf{x}).$$

3.6 Linear Regression as a GLM

Suppose $y|\mathbf{x};\boldsymbol{\theta} \sim \mathcal{N}(\mu, \sigma^2)$. From the exponential family form of the Gaussian distribution function, we know that

$$\eta = \mu.$$

Using the GLM assumptions,

$$h_{\boldsymbol{\theta}}(\mathbf{x}) = E(y|\mathbf{x};\boldsymbol{\theta}) = \mu = \eta = \boldsymbol{\theta}^\top \mathbf{x}.$$

So we recover exactly the linear regression hypothesis:

$$h_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta}^\top \mathbf{x}.$$

Proposition 3.6.1. Ordinary least squares is a GLM obtained by choosing the Gaussian family and using the identity response function.

For the Gaussian family,

$$g(\eta) = \eta.$$

So the canonical response function is simply the **identity function**.

3.7 Logistic Regression as a GLM

Now suppose $y \in \{0, 1\}$, so it is natural to model

$$y|\mathbf{x};\boldsymbol{\theta} \sim \text{Bernoulli}(\phi).$$

From the Bernoulli exponential family form, we know

$$\phi = \frac{1}{1 + e^{-\eta}}.$$

Also, for a Bernoulli random variable,

$$E(y|\mathbf{x};\boldsymbol{\theta}) = \phi.$$

Therefore,

$$h_{\boldsymbol{\theta}}(\mathbf{x}) = E(y|\mathbf{x};\boldsymbol{\theta}) = \phi = \frac{1}{1 + e^{-\eta}} = \frac{1}{1 + \exp(-\boldsymbol{\theta}^\top \mathbf{x})}.$$

So we recover the logistic regression hypothesis:

$$h_{\boldsymbol{\theta}}(\mathbf{x}) = \frac{1}{1 + \exp(-\boldsymbol{\theta}^\top \mathbf{x})}.$$

Proposition 3.7.1. Logistic regression is a GLM obtained by choosing the Bernoulli family. The sigmoid appears naturally as the canonical response function.

For the Bernoulli family,

$$g(\eta) = \frac{1}{1 + e^{-\eta}}.$$

So the canonical response function is the **logistic function**.

3.8 Why GLMs matter?

GLMs are useful because they turn model design into a principled recipe. Instead of guessing a model directly:

- first choose a suitable distribution for the output;
- then derive the hypothesis function from exponential family structure.

This is conceptually powerful because it explains where common models come from, rather than presenting them as isolated formulas.

Example intuition

- If y is continuous, Gaussian is natural $\implies$ linear regression.
- If y is binary, Bernoulli is natural $\implies$ logistic regression.
- If y is count data, Poisson is natural $\implies$ Poisson regression.

3.9 General Recipe for constructing a GLM

Algorithm 5 Recipe for constructing a GLM

Require: prediction problem with output y and features $\mathbf{x}$
  Choose a distribution for $y|\mathbf{x}$ from the exponential family
  Identify its natural parameter η
  Compute the conditional mean $E(y|\mathbf{x})$
  Assume $\eta = \boldsymbol{\theta}^\top \mathbf{x}$
  Express $h_{\boldsymbol{\theta}}(\mathbf{x}) = E(y|\mathbf{x})$ as a function of $\boldsymbol{\theta}^\top \mathbf{x}$
  return a generalised linear model $h_{\boldsymbol{\theta}}$

Practical interpretation

This is not merely a theorejm; it is a design template for building probabilistic prediction models.

3.10 Geometric and Statistical View

Statistical view

GLMs model the output through a distributional assumption:

$$p(y|\mathbf{x}; \boldsymbol{\theta}).$$

Geometric view

The complexity of the model comes from applying a response function to the linear predictor $\boldsymbol{\theta}^\top \mathbf{x}$. So the GLMs remain "linear" at the parameter-input level, even if the final prediction is nonlinear. This explains why linear regression outputs a linear function and logistic regression outputs a nonlinear sigmoid of a linear score.

4 Generative Models

4.1 Overview

So far, many models we studied were **discriminative**: they model the conditional distribution $p(y|\mathbf{x})$, or directly learn a mapping from $\mathbf{x}$ to y. Logistic regression is the main example. In contrast, **generative learning algorithms** model $p(\mathbf{x}|y)$ and $p(y)$, and then recover $p(y|\mathbf{x})$ using Bayes' Theorem.

Definition 4.1.1 (Generative vs discriminative models)**.** A **discriminative model** learns $p(y|\mathbf{x})$ directly. A **generative model** learns $p(\mathbf{x}|y)$ and $p(y)$, then computes

$$p(y|\mathbf{x}) = \frac{p(\mathbf{x}|y)p(y)}{p(\mathbf{x})}.$$

Since $p(\mathbf{x})$ does not depend on the class label when making a prediction, classification can be done by

$$\underset{y}{\operatorname{argmax}} p(y|\mathbf{x}) = \underset{y}{\operatorname{argmax}} p(\mathbf{x}|y)p(y).$$

Main intuition

A generative classifier builds a separate model for each class, then asks: *Which class would be more likely to have generated this input?*This viewpoint is especially natural in problems such as:

- classifying continuous feature vectors with class-conditional Gaussians;
- classifying text with class-conditional word distributions.

4.2 Gaussian Discriminant Analysis (GDA)

When the input features are continuous-valued, we introduce **Gaussian Discriminant Analysis** (GDA). The key assumption is that the class-conditional distribution $p(\mathbf{x}|y)$ is multivariate Gaussian.

Definition 4.2.1 (Multivariate Gaussian)**.** A d-dimensional random vector $\mathbf{X}$ has multivariate Gaussian distribution $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$ if its density if

$$p(\mathbf{x}; \boldsymbol{\mu}, \Sigma) = \frac{1}{\sqrt{(2\pi)^d |\Sigma|}} \exp\left[-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right],$$

where:

- $\boldsymbol{\mu}$ is the mean vector,
- Σ is the covariance matrix,
- $|\Sigma|$ is the determinant of Σ.

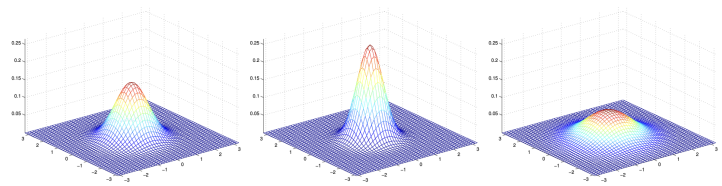


Figure 1: Examples of multivariate Gaussian distribution functions.

Basic properties

If $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$, then $E(\mathbf{X}) = \boldsymbol{\mu}$ and $\mathrm{Cov}(\mathbf{X}) = \Sigma$.

Geometric intuition

The density contours of multivariate Gaussian are ellipses (or ellipsoids in higher dimensions). Figures below are examples generated by varying Σ:

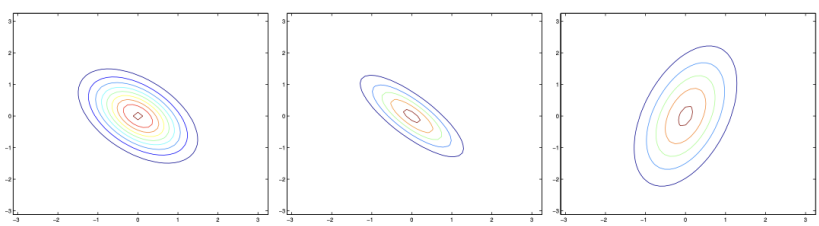


Figure 2: Example Gaussian distribution contours generated by varying Σ.

The plots in Figure 2 used, respectively

$$\Sigma = \begin{bmatrix} 1 & -0,5 \\ -0.5 & 1 \end{bmatrix}, \quad \Sigma = \begin{bmatrix} 1 & -0.8 \\ -0.8 & 1 \end{bmatrix}, \quad \Sigma = \begin{bmatrix} 3 & 0.8 \\ 0.8 & 1 \end{bmatrix}.$$

- Larger covariance makes the distribution more spread out;
- nonzero off-diagonal terms tilt and stretch the ellipse;
- changing $\boldsymbol{\mu}$ shifts the centre of the density.

This matters because GDA models each class using such Gaussian contours.

Definition 4.2.2 (Gaussian Discriminant Analysis)**.** For binary classification with $y \in \{0, 1\}$, GDA assumes $y \sim \text{Bernoulli}(\phi)$, and

$$\mathbf{x}|y = 0 \sim \mathcal{N}(\boldsymbol{\mu}_0, \Sigma), \quad \mathbf{x}|y = 1 \sim \mathcal{N}(\boldsymbol{\mu}_1, \Sigma).$$

Thus:

- $\phi = p(y = 1)$ is the class prior,
- $\boldsymbol{\mu}_0$ and $\boldsymbol{\mu}_1$ are class-specific means,
- Σ is a **shared covariance matrix**.

Interpretation

Each class gets its own Gaussian centre, but both classes share the same covariance shape. So the model says:

- class 0 examples cluster around $\boldsymbol{\mu}_0$,
- class 1 examples cluster around $\boldsymbol{\mu}_0$,
- both clouds have the same geometric spread pattern.

4.3 Maximum Likelihood Estimation in GDA

Given training data $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, the log-likelihood is

$$\ell(\phi, \boldsymbol{\mu}_0, \boldsymbol{\mu}_1, \Sigma) = \sum_{i=1}^n \log p(\mathbf{x}^{(i)}, y^{(i)}; \phi, \boldsymbol{\mu}_0, \boldsymbol{\mu}_1, \Sigma).$$

Maximising this gives the MLEs:

$$\begin{aligned}
\phi &= \frac{1}{n} \sum_{i=1}^n \mathbf{1}_{\{y^{(i)}=1\}}, \\
\boldsymbol{\mu}_0 &= \frac{\sum_{i=1}^n \mathbf{1}_{\{y^{(i)}=0\}} \mathbf{x}^{(i)}}{\sum_{i=1}^n \mathbf{1}_{\{y^{(i)}=0\}}} \\
\boldsymbol{\mu}_1 &= \frac{\sum_{i=1}^n \mathbf{1}_{\{y^{(i)}=1\}} \mathbf{x}^{(i)}}{\sum_{i=1}^n \mathbf{1}_{\{y^{(i)}=1\}}}, \\
\Sigma &= \frac{1}{n} \sum_{i=1}^n (\mathbf{x}^{(i)} - \boldsymbol{\mu}_{y^{(i)}})(\mathbf{x}^{(i)} - \boldsymbol{\mu}_{y^{(i)}})^\top.
\end{aligned}$$

These are exactly the sample class prior, class means, and pooled covariance estimate.

Proposition 4.3.1. In GDA, the MLE parameters have simple closed-form expressions.

Unlike logistic regression, GDA does not require iterative optimisation for parameter estimation. We can estimate all parameters directly from empirical class statistics.

4.4 Prediction in GDA

Once ϕ, $\boldsymbol{\mu}_0$, $\boldsymbol{\mu}_1$, and Σ are estimated, we classify a new point $\mathbf{x}$ by comparing

$$p(\mathbf{x}|y=1)p(y=1) \quad \text{and} \quad p(\mathbf{x}|y=0)p(y=0).$$

That is,

$$\hat{y} = \operatorname*{argmax}_{y \in \{0,1\}} p(\mathbf{x}|y)p(y).$$

Geometric interpretation

In the example shown in Figure 3, we can see two Gaussian contour families centred at different means; and the decision boundary is a straight line where $p(y=1|\mathbf{x}) = 0.5$.

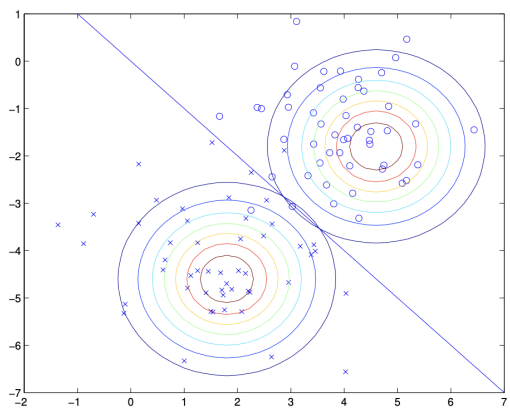


Figure 3: The training set, as well as the contours of the two Gaussian distributions that have been fit to the data in each of the two classes.

Proposition 4.4.1. Under the shared-covariance GDA assumptions, the decision boundary is linear in $\mathbf{x}$.

This is why GDA, despite being generative, still produces a linear classifier.

4.5 Relationship between GDA and Logistic Regression

Under the GDA assumptions,

$$p(y=1|\mathbf{x}) = \frac{1}{1+\exp(-\boldsymbol{\theta}^\top \mathbf{x})}$$

for an appropriate parameter vector $\boldsymbol{\theta}$ determined by ϕ, $\boldsymbol{\mu}_0$, $\boldsymbol{\mu}_1$ and Σ.

So the posterior under GDA has exactly the same **logistic form** as logistic regression.

Proposition 4.5.1. If $p(\mathbf{x}|y)$ is Gaussian with shared covariance, then $p(y|\mathbf{x})$ is logistic.

NOTE: The converse of Proposition 4.5.1 is not true: logistic posterior form does **not** imply Gaussian class-conditionals. So GDA makes **stronger assumptions** than logistic regression.

Practical trade-off

- **GDA**: stronger assumptions, more data-efficient if assumptions are correct.
- **Logistic Regression**: weaker assumptions, usually more robust when the Gaussian assumption is wrong.

Takeaway

GDA can be better when its probabilistic assumptions are approximately correct; logistic regression is usually safer when they are not.

4.6 Naive Bayes

GDA is for continuous-valued features. For discrete-valued features, we introduce **Naive Bayes**. The motivating application is text classification, such as spam detection.

Setup

Suppose:

- $y \in \{0, 1\}$ is the class label,
- $\mathbf{x}$ is a binary feature vector,
- $x_j = 1$ indicates that word j appears in the document.
- $x_j = 0$ otherwise.

If the vocabulary size is d, then $\mathbf{x} \in \{0, 1\}^d$.

4.7 Naive Bayes Assumption

Directly modelling the full joint $p(\mathbf{x}|y)$ is impossible in high dimensions because the number of possible feature vectors is exponential in d.

So Naive Bayes makes the simplifying assumption that the features are **conditionally independent given the class**.

Definition 4.7.1 (Naive Bayes assumption)**.** For $j = 1, \ldots, d$,

$$p(\mathbf{x}|y) = \prod_{i=1}^{d} p(x_j|y).$$

Interpretation

Knowing the class label y, the model assumes that one feature tells us nothing further about another feature.

For text, this means: *Once we know whether an email is spam or not, the presence of one word is assumed independent of the presence of another word.* This assumption is obviously unrealistic, but the resulting classifier often works surprisingly well in practice.

4.8 Bernoulli Naive Bayes

For binary features, Naive Bayes uses Bernoulli distributions. Let

- $\phi_y = p(y = 1)$,
- $\phi_{j|y=1} = p(x_j = 1|y = 1)$,
- $\phi_{j|y=0} = p(x_j = 1|y = 0)$.

Given the training set $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, we can write down the joint likelihood of the data:

$$L(\phi_y, \phi_{j|y=0}, \phi_{j|y=1}) = \prod_{i=1}^n p(\mathbf{x}^{(i)}, y^{(i)}).$$

Maximising with respect to $\phi_y, \phi_{j|y=0}, \phi_{j|y=1}$ gives the MLEs:

$$\begin{aligned}
\phi_{j|y=1} &= \frac{\sum_{i=1}^n \mathbf{1}_{\{x_j^{(i)}=1 \wedge y^{(i)}=1\}}}{\sum_{i=1}^n \mathbf{1}_{\{y^{(i)}=1\}}}, \\
\phi_{j|y=0} &= \frac{\sum_{i=1}^n \mathbf{1}_{\{x_j^{(i)}=1 \wedge y^{(i)}=0\}}}{\sum_{i=1}^n \mathbf{1}_{\{y^{(i)}=0\}}}, \\
\phi_y &= \frac{1}{n} \sum_{i=1}^n \mathbf{1}_{\{y^{(i)}=1\}}.
\end{aligned}$$

These are simple empirical frequencies.

Prediction rule

For a new feature vector $\mathbf{x}$, compute

$$p(y = 1|\mathbf{x}) = \frac{p(\mathbf{x}|y = 1)p(y = 1)}{p(\mathbf{x}|y = 1)p(y = 1) + p(\mathbf{x}|y = 0)p(y = 0)},$$

where

$$p(\mathbf{x}|y) = \prod_{j=1}^d p(x_j|y).$$

Then choose the class with larger posterior probability.

4.9 Laplacing Smoothing

A major practical issue arises when a word or feature never appears in one class in the training set.

Then the MLE may assign probability zero:

$$\phi_{j|y} = 0.$$

This causes the entire product $p(\mathbf{x}|y)$ to become zero, which is disastrous for prediction.

Definition 4.9.1 (Laplace smoothing)**.** For a multinomial parameter ϕ_j, Laplace smoothing replaces

$$\phi_j = \frac{\text{count}_j}{n}$$

with

$$\phi_j = \frac{1 + \text{count}_j}{k + n}.$$

, where k is the number of possible outcomes.

For Bernoulli Naive Bayes

$$\phi_{j|y=1} = \frac{1 + \sum_{i=1}^{n} \mathbf{1}_{\{x_j^{(i)}=1 \wedge y^{(i)}=1\}}}{2 + \sum_{i=1}^{n} \mathbf{1}_{\{x_j^{(i)}=0 \wedge y^{(i)}=1\}}},$$

$$\phi_{j|y=0} = \frac{1 + \sum_{i=1}^{n} \mathbf{1}_{\{x_j^{(i)}=1 \wedge y^{(i)}=0\}}}{2 + \sum_{i=1}^{n} \mathbf{1}_{\{y^{(i)}=0\}}}.$$

This ensures that no estimated probability is exactly zero.

Why it matters

Laplace smoothing is often essential for good practical performance, especially in text classification.

4.10 Multinomial Event Model for Text

A different event model for text classification, which often performs better than Bernoulli Naive Bayes.

Bernoulli event model

- Each dictionary word is a binary feature;
- only records whether a word appears or not.

Multinomial event model

Now let $x_j \in \{1, \ldots, |V|\}$ denote the identity of the jth word in the document, where $|V|$ is the vocabulary size. So a document of length d is represented as

$$(x_1, x_2, \ldots, x_d).$$

The model assumes:

1. first sample the class y,
2. then generate each word independently from a class-specific multinomial distribution.

Parameters

$$\phi_{k|y=1} = p(x_j = k|y = 1), \quad \phi_{k|y=0} = p(x_j = k|y = 0), \quad \phi_y = p(y = 1).$$

If we are given with a training set $\{(\mathbf{x}^{(i)}, y^{(i)})\}$ where $\mathbf{x}^{(i)} = \begin{bmatrix} x_1^{(i)} & \cdots & x_{d_i}^{(i)} \end{bmatrix}^\top$ (here d_i is the number of words in the ith training example), then the likelihood of the data:

$$\begin{aligned} L(\phi_y, \phi_{k|y=0}, \phi_{k|y=1}) &= \prod_{i=1}^{n} p(\mathbf{x}^{(i)}, y^{(i)}) \\ &= \prod_{i=1}^{n} \left[\prod_{j=1}^{d_i} p(x_j^{(i)}|y; \phi_y, \phi_{k|y=0}, \phi_{k|y=1}) \right] p(y^{(i)}; \phi_y). \end{aligned}$$

Maximising this with respect to the parameters yields the MLEs:

$$\begin{aligned} \phi_{k|y=1} &= \frac{\sum_{i=1}^{n} \sum_{j=1}^{d_i} \mathbf{1}_{\{x_j^{(i)}=k \wedge y^{(i)}=1\}}}{\sum_{i=1}^{n} \mathbf{1}_{\{y^{(i)}=1\}} d_i}, \\ \phi_{k|y=0} &= \frac{\sum_{i=1}^{n} \sum_{j=1}^{d_i} \mathbf{1}_{\{x_j^{(i)}=k \wedge y^{(i)}=0\}}}{\sum_{i=1}^{n} \mathbf{1}_{\{y^{(i)}=0\}} d_i}, \\ \phi_y &= \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}_{\{y^{(i)}=1\}}. \end{aligned}$$

These count total word occurrences in each class, not just document-level presence/absence.

With Laplace smoothing

Add 1 to each numerator and $|V|$ to each denominator:

$$\begin{aligned} \phi_{k|y=1} &= \frac{1 + \sum_{i=1}^{n} \sum_{j=1}^{d_i} \mathbf{1}_{\{x_j^{(i)}=k \wedge y^{(i)}=1\}}}{|V| + \sum_{i=1}^{n} \mathbf{1}_{\{y^{(i)}=1\}} d_i}, \\ \phi_{k|y=0} &= \frac{1 + \sum_{i=1}^{n} \sum_{j=1}^{d_i} \mathbf{1}_{\{x_j^{(i)}=k \wedge y^{(i)}=0\}}}{|V| + \sum_{i=1}^{n} \mathbf{1}_{\{y^{(i)}=0\}} d_i}. \end{aligned}$$

Practical distinction

- **Bernoulli Naive Bayes** cares whether a word appears.
- **Multinomial Naive Bayes** cares how many times words appear.

4.11 GDA vs Naive Bayes

These are both generative models, but they are used in different situations.

GDA

- Feature vector $\mathbf{x}$ is continuous;
- model $p(\mathbf{x}|y)$ with a multivariate Gaussian.

Naive Bayes

- Features are discrete or discretised;
- model each feature independently conditioned on y.

Practical Note

When continuous attributes are not well modelled by a multivariate Gaussian, discretising them and using Naive Bayes may work better than GDA.

4.12 Advantages and Limitations

Strengths of generative models

- Probabilistically interpretable;
- often simple to estimate;
- can be data-efficient when assumptions are correct;
- Naive Bayes is fast, easy to implement, and often a strong baseline.

Limitations

- GDA depends on Gaussian assumptions;
- Naive Bayes depends on conditional independence assumptions;
- both may fail when their modelling assumptions are badly violated.

5 k-Nearest Neighbours

5.1 Overview

k-Nearest Neighbours (kNN) is a simple supervised learning method for **classification** and **regression**.

Unlike linear regression or logistic regression, kNN does **not** fit an explicit parametric model during training. Instead, it stores the training data and makes predictions by looking at the training points nearest to a query point.

Main idea

Given a new input $\mathbf{x}$:

1. compute its distance to all training examples;
2. find the k nearest neighbours;
3. predict using those neighbours.

Why kNN matters

kNN is important conceptually because it shows that learning does not always mean fitting a parameter vector $\boldsymbol{\theta}$ or $\mathbf{w}$. Instead, prediction can be based directly on the geometry of the data.

5.2 Problem Setup

Let the training set be $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{n}$, where $\mathbf{x}^{(i)} \in \mathbb{R}^d$, $y^{(i)}$ is either a class label (classification) or a real number (regression). Given a query point $\mathbf{x}$, kNN predicts based on the set of its k nearest training points.

5.3 Distance Metrics

The notion of "nearest" depends on the chosen distance function.

Definition 5.3.1 (Euclidean distance)**.** For $\mathbf{x}, \mathbf{z} \in \mathbb{R}^d$,

$$d(\mathbf{x}, \mathbf{z}) = \|\mathbf{x} - \mathbf{z}\|_2 = \sqrt{\sum_{j=1}^{d} (x_j - z_j)^2}.$$

This is the most common choice.

Other common distances

Manhattan distance

$$d(\mathbf{x}, \mathbf{z}) = \|\mathbf{x} - \mathbf{z}\|_1 = \sum_{j=1}^{d} |x_j - z_j|.$$

General Minkowski distance

$$d(\mathbf{x}, \mathbf{z}) = \left(\sum_{j=1}^{d} |x_j - z_j|^p \right)^{1/p}.$$

Practical note

The performance of kNN depends strongly on the distance metric, because the entire method is based on geometric proximity.

5.4 The kNN Rule

Definition 5.4.1 (Set of k nearest neighbours)**.** For a query point $\mathbf{x}$, let $\mathcal{N}_k(\mathbf{x})$ denote the set of indices of the k training examples closest to $\mathbf{x}$ under the chosen distance metric.

5.5 kNN for Classification

For classification, kNN predicts the label that appears most frequently among the k nearest neighbours.

Definition 5.5.1 (kNN Classifier)**.** Given $\mathcal{N}_k(\mathbf{x})$, the kNN classification rule is

$$\hat{y} = \operatorname*{argmax}_{c} \sum_{i \in \mathcal{N}_k(\mathbf{x})} \mathbf{1}_{\{y^{(i)} = c\}}.$$

So the predicted class is the **majority vote** among the nearest neighbours.

Special case: $k = 1$

When $k = 1$, the prediction is simply the label of the single nearest training point.

Interpretation

A query point is assigned the class most common in its local neighbourhood.

5.6 kNN for Regression

For regression, kNN predicts by averaging the target values of the k nearest neighbouts.

Definition 5.6.1 (kNN Regressor)**.** Given $\mathcal{N}_k(\mathbf{x})$, the kNN regression rule is

$$\hat{y} = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x})} y^{(i)}.$$

So the prediction is the local average of nearby outputs.

Weighted variant

Sometimes closer neighbouts are given more influence, for example

$$\hat{y} = \frac{\sum_{i \in \mathcal{N}_k(\mathbf{x})} w_i y^{(i)}}{\sum_{i \in \mathcal{N}_k(\mathbf{x})} w_i},$$

where a common choice is

$$w_i = \frac{1}{d(\mathbf{x}, \mathbf{x}^{(i)}) + \varepsilon}.$$

This often improves performance by making very close points count more.

5.7 Algorithm

Algorithm 6 kNN Prediction

Require: training set $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{n}$, query point $\mathbf{x}$, number of neighbours k, distance metric $d(\cdot, \cdot)$, task $\in \{\text{regression}, \text{classification}\}$

```
P ← empty map
for i ∈ {1, ..., n} do
    P.put(x^(i), d(x, x^(i)))
end for
Sort the keys of P by values in ascending order
N_k(x) ← indices of first k keys of P.
if task = classification then
    return argmax_c Σ_{i∈N_k(x)} 1_{{y^(i)=c}}
end if
if task = regression then
    return (1/k) Σ_{i∈N_k(x)} y^(i)
end if
```

Observation

There is effectively **no training phase** beyond storing the data. Most of the computational work is deferred to prediction time.

5.8 Geometric Intuition

kNN is a purely geometric learning method.

Classification view

The feature space is partitioned into regions according to which nearby training labels dominate locally.

For $k = 1$, the decision regions are determined by a **Voronoi partition**: every point in the region of a training sample is classified the same way as that sample. For larger k, the boundary becomes smoother because more neighbours are averaged into the decision.

Regression view

The prediction at $\mathbf{x}$ is a local average, so kNN regression can be viewed as a form of local smoothing.

Key intuition

kNN assumes that **nearby points tend to have similar outputs**. This is a locality assumption:

- close points should have similar labels,
- or close points should have similar target values.

5.9 Why kNN is Non-Parametric

Definition 5.9.1 (Non-parametric model)**.** A learning alforithm is called **non-parametric** if the complexity of the learned representation grows with the size of the training data, rather than being summarised by a fixed-dimensional parameter vector.

Proposition 5.9.2. kNN is a non-parametric learning algorithm.

kNN does not compress the training data into a fixed parameter vector $\boldsymbol{\theta}$. Instead, it keeps the training set and uses it directly at prediction time.

This is similar in spirit to locally weighted regression:

- both are local methods,
- both depend directly on stored training examples.

5.10 Choice of k

The hyperparameter k controls the smoothness of the prediction.

	Small k	**Large k**
Bias	Low	High
Variance	High	Low
Noise/Outliers	Sensitive to noise	Smoother predictions
Overfit/Underfit	May overfit	May underfit

Table 1: Comparison between the choices of the hyperparameter k.

Proposition 5.10.1. The choice of k induces a bias-variance tradeoff.

Intuition

- $k = 1$: extremely local, very flexible, high noisy;
- very large k: overly global, too smooth, may ignore local structure.

So k is usually selected by validation or cross-validation.

5.11 Importance of Feature Scaling

Because kNN relies entirely on distances, the scale of each feature matters greatly.

Suppose one feature ranges from 0 to 1, while another ranges from 0 to 10^5. Then the second feature may dominate Euclidean distance, even if it is not more important for prediction.

Practical rule

Before applying kNN, features should usually be standardised or normalised. Common approaches:

- z-score standardisation:

$$x_j^{(i)} \leftarrow \frac{x_j^{(i)} - \mu_j}{\sigma_j}$$

- min-max normalisation:

$$x_j^{(i)} \leftarrow \frac{x_j^{(i)} - \min\{x_j^{(i)}\}_{i=1}^n}{\max\{x_j^{(i)}\}_{i=1}^n - \min\{x_j^{(i)}\}_{i=1}^n}$$

Common pitfall

Running kNN on unscaled data often leads to misleading neighbour relationships.

5.12 Curse of Dimensionality

One major limitation of kNN is that it performs poorly in high dimensions.

Informal phenomenon

As dimension increases:

- distances between points become less informative,
- nearest and farthest neighbours become more similar in distance,
- local neighbourhoods become sparse.

This makes "nearest" much less meaningful.

Consequence

kNN often works best in:

- low-dimensional problems,
- or after dimensionality reduction or careful feature engineering.

Takeaway

kNN suffers badly from the curse of dimensionality because locality becomes unreliable in high-dimensional spaces.

5.13 Computational Complexity

Let n be the number of training examples, d be the number of features. The training time is $O(1)$ if we ignore the cost of storing data.

To classify one query point, compute distance to all n points in $O(nd)$, then find the nearest k. So naive prediction is roughly $O(md)$ per query, or $O(nd + n \log n)$ if a full sort is used.

The space complexity if $O(nd)$, since all training data must be stored.

Practical implication

kNN is easy to train but can be expensive at inference time.

5.14 Weighted kNN and Local Smoothing

A useful extension is to weight neighbours by distance.

Definition 5.14.1 (Distance-weighted kNN)**.** Instead of treating all k neighbours equally, assign weights

$$w_i = K(d(\mathbf{x}, \mathbf{x}^{(i)})),$$

where K decreases with distance.

For example,

$$w_i = \exp\left(- \frac{\|\mathbf{x} - \mathbf{x}^{(i)}\|_2^2}{2\tau^2} \right).$$

Then weighted classification uses weighted votes, where weighted regression uses weighted averages.

Connection

This makes kNN conceptually close to **locally weighted regression**: both emphasise nearby points more heavily.

5.15 Statistical Interpretation

kNN can be viewed as a local approximation to the conditional distribution.

For classification

The fraction of class-c neighbours near $\mathbf{x}$ approximates

$$P(y = c|\mathbf{x}).$$

For regression

The local average approximates

$$E(y|\mathbf{x}).$$

So kNN is not just a heuristic; it is a local estimator of the target conditional structure.

5.16 Strengths and Limitations

Strengths

- Simple and intuitive;
- no explicit training optimisation;
- naturally handles nonlinear decision boundaries;
- can work well on small, low-dimensional dataset;
- useful as a baseline.

Limitations

- Prediction can be slow;
- sensitive to feature scaling;
- sensitive to irrelevant features,
- choice of k matters a lot;
- performs poorly in high dimensions.

kNN vs Logistic Regression

- **kNN**: non-parametric, local, nonlinear boundary possible.
- **Logistic regression**: parametric, global, linear boundary in feature space.

kNN vs GDA / Naive Bayes

- **kNN**: makes few probabilisitc assumptions.
- **Generative models**: assume explicit distributions such as Gaussians or conditional independence.

kNN vs Decision Trees

- **kNN**: neighbourhood-based.
- **Decision trees**: partition feature space by axis-aligned splits.

Main comparison

kNN is flexible because it avoids strong modelling assumptions, but that flexibility comes with computational and statistical costs.

6 Decision Trees

6.1 Overview

Decision trees are supervised learning models for **classification** and **regression** that make predictions by recursively partitioning the input space into regions $R_1, \ldots, R_k$, and assigning one prediction w_j to each corresponding region R_j.

Ensemble methods build on this idea by combining many trees:

- **bagging** and **random forests** mainly reduce variance;
- **boosting** mainly reduces bias.

So:

$$\text{single tree} \to \text{many} \to \text{better generalisation.}$$

6.2 Model Formulation

Definition 6.2.1 (Decision tree as piecewise-constant predictor)**.** A decision tree partitions the input space into k disjoint regions $R_1, R_2, \ldots, R_k$ whose union is the entire input domain, and predicts a constant value w_j on each region R_j. Thus,

$$f(\mathbf{x}) = \sum_{j=1}^{k} w_j \mathbf{1}_{\{\mathbf{x} \in R_j\}}.$$

Interpretation

A tree is therefore a **piecewise-constant model**:

- for regression, each leaf predicts a number;
- for classification, each leaf predicts a class or class distribution.

6.3 Tree Representation

A tree implements this partition recursively. Each internal node contains a split of the form

$$x_j \geq \theta \quad \text{or equivalently} \quad x_j < \theta,$$

where $j \in \{1, \ldots, d\}$ is a feature index, $\theta \in \mathbb{R}$ is a threshold. We may assume the standard setting where each node branches using a single inequality on a single feature, which implies that the resulting regions are **axis-aligned**.

Geometric implication

Because each split cuts along one coordinate axis, the final regions are axis-aligned rectangles (or hyperrectangles in higher dimensions).

6.4 Prediction at Leaves

Let a leaf correspond to region R_j.

Regression

A standard choice is the average label in that region:

$$w_j = \frac{\sum_{i=1}^{n} y^{(i)} \mathbf{1}_{\{\mathbf{x}^{(i)} \in R_j\}}}{\sum_{i=1}^{n} \mathbf{1}_{\{\mathbf{x}^{(i)} \in R_j\}}}.$$

Classification

A standard choice is the majority class among training points in that leaf.

Interpretation

Leaves summarise local subsets of the training data:

- regression leaves return local averages;
- classification leaves return local majority votes.

6.5 Fitting a Tree

Let the training set be $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{n}$. The training loss can be written as

$$L(f) = \sum_{i=1}^{n} \ell(y^{(i)}, f(\mathbf{x}^{(i)})) = \sum_{j=1}^{n} \sum_{\mathbf{x}^{(i)} \in R_j} \ell(y^{(i)}, w_j).$$

Unlike linear or logistic regression, this optimisation is **not smooth** because the tree structure is discrete. The calculus-based optimisation is not directly useful here, and finding the globally optimal tree is NP-complete. So in practice we use a **greedy recursive algorithm**.

6.6 Greedy Splitting

At a node containing dataset S, we consider candidate splits of the form

$$x_j \geq \theta.$$

We choose the split maximising a gain function:

$$(j, \theta) = \operatorname*{argmax}_{j,\theta} G(j, \theta).$$

Why this is computationally feasible

For a fixed feature j, only finite many thresholds matter. If we sort the feature values

$$x_j^{(i_1)} \leq x_j^{(i_2)} \leq \cdots \leq x_j^{(i_n)},$$

then only the $n-1$ gaps between consecutive sorted values produce distinct splits. A common candidate threshold is the midpoint

$$\theta = \frac{x_j^{(i_\ell)} + x_j^{(i_{\ell+1})}}{2}.$$

So for each node we search over roughly $d(n-1)$ candidate splits.

6.7 Gain and Cost

Let's introduce an abstract node cost function $C(\cdot)$, then define gain as cost reduction after splitting. If a parent node has sample set S, and a split produces left and right children L and R, then

$$G(j,\theta) = C(S) - \left[\frac{|L|}{|S|}C(L) + \frac{|R|}{|S|}C(R)\right].$$

Interpretation

A good split is one that makes the children much "better" than the parent. Equivalently, maximising gain is the same as minimising the weighted post-split cost.

6.8 Split Criteria for Regression and Classification

Regression trees

A standard node cost is MSE:

$$C(S) = \frac{1}{|S|}\sum_{\mathbf{x}^{(i)}\in R}(y^{(i)} - w)^2.$$

Classification trees

A standard node cost is entropy:

$$C(S) = -\sum_{c\in\mathcal{C}} p_c \log p_c$$

where p_c is the fraction of points in S belonging to class c.

In practice, another common classification cost is **Gini impurity**:

$$C(S) = 1 - \sum_{c\in\mathcal{C}} p_c^2.$$

This is widely used in CART-style implementations.

6.9 Algorithm

Algorithm 7 Greedy decision tree fitting

```
Require: training set D at current node
  if stopping criterion is met then
      Create a leaf
      Output mean label (regression) or majority class (classification)
  else
      for each feature j do
          for each candidate threshold θ do
              Compute gain G(j, θ)
          end for
      end for
      Choose split (j, θ) ← argmax_{j,θ} G(j, θ)
      Partition D into left and right subsets
      Recursively fit left subtree
      Recursively fit right subtree
  end if
```

Key feature

This is a **top-down recursive greedy algorithm**:

- split the current node as well as possible;
- then repeat on each child.

6.10 Regularisation and Pruning

If allowed to grow arbitrarily deep, a decision tree can fit any internally consistent training set perfectly. This creates two problems:

- overfitting,
- model size explosion.

Common regularisation methods

Several practical heuristics:

- limit maximum depth;
- limit number of leaves;
- require minimum samples to split;
- stop splitting if gain is below a threshold.

Definition 6.10.1 (Pruning)**.** Pruning means removing branches whose added complexity is not worth the performance gain.

Pruning is a post-processing regularisation strategy: grow a big tree, then merge nodes back if they do not justify their complexity.

Bias-variance view

- shallow trees: high bias, low variance
- deep trees: low bias, high variance.

6.11 Strengths and Weaknesses

Strengths

- Highly interpretable;
- robust to outliers;
- handle mixtures of continuous and discrete features;
- robust to monotone transformations of inputs;
- can achieve a decent fit quickly.

Weaknesses

- Often generalise poorly even when regularised;
- highly unstable near boundaries.

Extra intuition

A single split can drastically change if the data changes slightly, which is why trees are high-variance learners. This instability is exactly why ensemble methods exploit.

7 Ensemble Methods

7.1 Overview

We can define ensemble as a weighted combination of models:

$$f(\mathbf{x}) = \sum_{i=1}^{m} \beta_i F_i(\mathbf{x}),$$

where $F_1, \ldots, F_m$ are base learners and β_i are weights. Often $\beta_i = 1/m$.

Interpretation

Instead of trusting one model, we average many similar models.

Main benefit

Ensemble learning tends to:

- reduce variance,
- improve robustness,
- improve predictive performance.

Main cost

- More computation,
- less interpretability than a single tree.

7.2 Bagging

Definition 7.2.1 (Bagging)**.** Bagging means **bootstrap aggregating**:

1. generate many bootstrap datasets by sampling the original training set with replacement;
2. train one model on each bootstrap dataset;
3. average their predictions (regression) or take a plurality vote (classification).

Why bagging works

If we trained all trees on exactly the same data with a deterministic training algorithm, the trees would be identical. Baggin injects randomness through the data.

Bootstrap fact

When a bootstrap sample has size n, the probability a particular original datapoint appears at least once is approximately

$$1 - e^{-1} \approx 0.632.$$

So each bootstrap sample uses only about 63.2% of the original examples on average,

Intuition

Each individual tree sees a slightly different dataset, so the ensemble is less sensitive to any one training sample.

Main effect

Bagging mainly **reduces variance**.

7.3 Random Forests

Definition 7.3.1 (Random forest)**.** A random forest is bagging applied to decision trees, with an extra source of randomness: at each split, only a random subset of features is considered. This is exactly choosing a subset of size $k < d$ at each node-splitting step.

Why this helps

Bagging alone makes trees different by varying the data. Random forests make them even less correlated by also varying the available features.

Interpretation

Each tree becomes:

- trained on different data,
- using different candidate features.

Practical effect

Random forests usually outperform plain bagging when trees are the base learner.

Takeaway

Bagging reduces variance by resampling data; random forests reduce it further by also randomising features.

7.4 Boosting

Boosting is very different from bagging. Instead of training models independently in parallel, boosting trains them **sequentially**, so that each new model is designed to improve the current ensemble.

The current ensemble after i steps can be defined as

$$f_i(\mathbf{x}) = \sum_{j=1}^{i} \beta_j F_j(\mathbf{x}).$$

Then the next learner is chosen based on the current model's weaknesses.

High-level difference

- Bagging / random forests: mostly reduce variance;
- boosting: mainly reduces bias.

Weak learner idea

Boosting is motivated by the idea that many weak learners can be combined into a strong learner.

7.5 Forward Stagewise Additive Modelling

We can also view boosting as building the ensemble step by step. At iteration i, we choose a new learner to improve $f_{i-1}(\mathbf{x})$ by adding a correction term. In parametric form, this can be written as choosing β_i, and model parameters θ_i, to minimise $L(f_{i-1} + \beta_i F_i)$.

Interpretation

Each step adds one more function to the ensemble:

- not starting over,
- just correcting what is currently wrong.

7.6 Gradient Boosting

The most important boosting method is **gradient boosting**. Instead of solving for the next function exactly, we can interpret the next learner as approximating a functional gradient descent step.

Let the current ensemble be f_{i-1}, and define the vector of current predictions on the training set. The gradient of the loss with respect to these predictions is

$$\mathbf{g} = \begin{bmatrix} \dfrac{\partial \ell(f_{i-1}(\mathbf{x}^{(i)}), y^{(1)})}{\partial f_{i-1}(\mathbf{x}^{(1)})} & \cdots & \dfrac{\partial \ell(f_{i-1}(\mathbf{x}^{(n)}), y^{(n)})}{\partial f_{i-1}(\mathbf{x}^{(n)})} \end{bmatrix}^\top.$$

Then fit a new model F_i to the **negative gradient targets** by solving

$$F_i = \operatorname*{argmin}_F \sum_{j=1}^{n} \Big(- g_j - F(\mathbf{x}^{(j)}) \Big)^2.$$

Then update:

$$f_i(\mathbf{x}) = f_{i-1}(\mathbf{x}) + \alpha F_i(\mathbf{x}),$$

where α is the learning rate.

Interpretation

At each step:

1. compute where the current model is wrong;

2. turn those errors into pseudo-residuals;
3. fit a weak learner to those pseudo-residuals;
4. add it into the ensemble.

This is the central idea of gradient boosting.

7.7 Gradient-Boosted Trees

In practice, the base learners in gradient boosting are almost always **heavily regularised decision trees**.

Why shallow trees work well

A shallow tree is:

- weak enough to avoid dominating ensemble,
- flexible enough to capture local structure.

So gradient boosting with trees gives:

- many weak trees,
- added together sequentially,
- producing a powerful nonlinear predictor.

This is often called **gradient tree boosting** or simply **gradient boosting** in practice.

7.8 XGBoost

XGBoost is a near real-world extension of gradient tree boosting. Its regularised objective:

$$L(f) = \left[\sum_{i=1}^{n} \ell(y^{(i)}, f(\mathbf{x}^{(i)})) \right] + \gamma J + \frac{1}{2}\lambda \sum_{j=1}^{J} w_j^2,$$

where:

- J is the number of leaves,
- w_j is the prediction value at leaf j,
- γ, λ are regularisation hyperparameters.

Similar to random forests, XGBoost uses feature subsampling. It also involves second-order approximation, using not only gradients but also Hessian information.

Why XGBoost matters

It is one of the most practically successful structured-data algorithms:

- tabular data,
- industry ML,
- Kaggle competitions.

XGBoost = gradient-boosted trees+better regularisation+better optimisation+better engineering

7.9 Comparison

Single tree

- Highly interpretable;
- high variance;
- often unstable.

Bagging / Random Forests

- Many trees in parallel;
- reduce variance;
- strong default choice for tabular data;
- less interpretable than one tree.

Boosting / Gradient Boosting

- Many trees sequentially;
- reduce bias and also often variance;
- often achieve the best predictive accuracy on structured data;
- more sensitive to hyperparameters.

One-line summary

Random forests stabilise trees; boosting strengthens trees.

8 Kernel Methods

8.1 Overview

Linear models are simple and computationally attractive, but they are limited to hypotheses that are linear in the input attributes. A standard way to increase expressiveness is to map the original input $\mathbf{x}$ into a new feature space via a feature map

$$\boldsymbol{\phi} : \mathcal{X} \to \mathbb{R}^p,$$

and then fit a linear model in the transformed space. The key idea of **kernel methods** is that we can often work in this high-dimensional feature space without explicitly constructing $\boldsymbol{\phi}$, provided we can efficiently compute inner products of the form

$$\langle \boldsymbol{\phi}(\mathbf{x}), \boldsymbol{\phi}(\mathbf{z}) \rangle.$$

This leads to the notion of a **kernel**.

8.2 Feature Maps

Suppose we want to fit a nonlinear function of scalar input x, for example a cubic

$$y = \theta_3 x^3 + \theta_2 x^2 + \theta_1 x + \theta_0.$$

Define the feature map

$$\boldsymbol{\phi}(x) = \begin{bmatrix} 1 \\ x \\ x^2 \\ x^3 \end{bmatrix}.$$

Then the cubic polynomial can be written as

$$\theta_3 x^3 + \theta_2 x^2 + \theta_1 x + \theta_0 = \boldsymbol{\theta}^\top \boldsymbol{\phi}(x).$$

A nonlinear function in the original input can be viewed as a linear function in a transformed feature space.

Definition 8.2.1 (Feature map)**.** A **feature map** is a function $\boldsymbol{\phi} : \mathcal{X} \to \mathbb{R}^p$ that maps the original input attributes to a new feature representation.

Interpretation

Kernel methods do not change linear learning itself; they change the representation on which linear learning operates.

8.3 Least Mean Squares in Feature Space

For ordinary least squares with hypothesis $h_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta}^\top \mathbf{x}$, batch gradient descent updates

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \sum_{i=1}^{n} \Big(\boldsymbol{\theta}^\top \mathbf{x}^{(i)} - y^{(i)} \Big) \mathbf{x}^{(i)}.$$

If instead we model

$$h_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta}^\top \boldsymbol{\phi}(\mathbf{x}),$$

then we simply replace each $\mathbf{x}^{(i)}$ by $\boldsymbol{\phi}(\mathbf{x}^{(i)})$, giving

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \sum_{i=1}^{n} \Big(\boldsymbol{\theta}^\top \boldsymbol{\phi}(\mathbf{x}^{(i)}) - y^{(i)} \Big) \boldsymbol{\phi}(\mathbf{x}^{(i)}).$$

The stochastic version becomes

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} = \alpha \Big(\boldsymbol{\theta}^\top \boldsymbol{\phi}(\mathbf{x}^{(i)}) - y^{(i)} \Big) \boldsymbol{\phi}(\mathbf{x}^{(i)}).$$

These are exactly the lifted LMS updates.

8.4 Why Explicit Feature Maps can be expensive?

If $\mathbf{x} \in \mathbb{R}^d$, and $\boldsymbol{\phi}(\mathbf{x})$ contains all monomials up to degree 3, then the lifted feature dimension is on the order of d^3.

Even though the original learning algorithm is linear, explicitly working with $\boldsymbol{\phi}(\mathbf{x})$ may be more expensive than the original nonlinear problem. This motivates the **kernel trick**.

8.5 Kernel Trick via Representer Form

The crucial observation is that if we initialise $\boldsymbol{\theta} = \mathbf{0}$ and update only by adding linear combinations of $\boldsymbol{\phi}(\mathbf{x}^{(i)})$, then $\boldsymbol{\theta}$ always lies in the span of the training feature vectors.

Proposition 8.5.1. At every iteration of batch gradient descent in feature space, the parameter vector can be written as

$$\boldsymbol{\theta} = \sum_{i=1}^{n} \beta_i \boldsymbol{\phi}(\mathbf{x}^{(i)})$$

for some coefficients $\beta_1, \ldots, \beta_n \in \mathbb{R}$.

Proof. We argue by induction. At initialisation,

$$\boldsymbol{\theta} = \mathbf{0} = \sum_{i=1}^{n} 0 \cdot \boldsymbol{\phi}(\mathbf{x}^{(i)}),$$

Assume that at some iteration,

$$\boldsymbol{\theta} = \sum_{i=1}^{n} \beta_i \boldsymbol{\phi}(\mathbf{x}^{(i)}).$$

Then the batch update gives

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \sum_{i=1}^{n} \left(\boldsymbol{\theta}^\top \boldsymbol{\phi}(\mathbf{x}^{(i)}) - y^{(i)} \right) \boldsymbol{\phi}(\mathbf{x}^{(i)}).$$

Substituting the inductive representation of $\boldsymbol{\theta}$,

$$\boldsymbol{\theta} = \sum_{i=1}^{n} \beta_i \boldsymbol{\phi}(\mathbf{x}^{(i)}) - \alpha \sum_{i=1}^{n} \left(\boldsymbol{\theta}^\top \boldsymbol{\phi}(\mathbf{x}^{(i)}) - y^{(i)} \right) \boldsymbol{\phi}(\mathbf{x}^{(i)}).$$

Grouping coefficients of $\boldsymbol{\phi}(\mathbf{x}^{(i)})$, we obtain another linear combination of the same vectors. Hence the claim remains true at the next iteration.

8.6 Kernalised LMS

Define the coefficients $\boldsymbol{\beta} = \begin{bmatrix} \beta_1 & \cdots & \beta_n \end{bmatrix}^\top$. If

$$\boldsymbol{\theta} = \sum_{j=1}^{n} \beta_j \boldsymbol{\phi}(\mathbf{x}^{(i)}),$$

then

$$\boldsymbol{\theta}^\top \boldsymbol{\phi}(\mathbf{x}^{(i)}) = \sum_{j=1}^{n} \beta_j \langle \boldsymbol{\phi}(\mathbf{x}^{(j)}), \boldsymbol{\phi}(\mathbf{x}^{(i)}) \rangle.$$

This motivates the following definition.

Definition 8.6.1 (Kernel function)**.** A **kernel** corresponding to a feature map $\boldsymbol{\phi}$ is the function

$$K(\mathbf{x}, \mathbf{z}) = \langle \boldsymbol{\phi}(\mathbf{x}), \boldsymbol{\phi}(\mathbf{z}) \rangle.$$

Then the coefficient update becomes

$$\beta_i \leftarrow \beta_i - \alpha \left(\sum_{j=1}^{n} \beta_j K(\mathbf{x}^{(j)}, \mathbf{x}^{(i)}) - y^{(i)} \right).$$

In vector form, if $K \in \mathbb{R}^{n \times n}$ is the kernel matrix with entries $K_{ij} = K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$, then

$$\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} - \alpha(K\boldsymbol{\beta} - \mathbf{y}).$$

Prediction

For a new point $\mathbf{x}$,

$$h(\mathbf{x}) = \boldsymbol{\theta}^\top \boldsymbol{\phi}(\mathbf{x}) = \sum_{i=1}^{n} \beta_i K(\mathbf{x}^{(i)}, \mathbf{x}).$$

So once the kernel is known, we never need to explicitly compute $\boldsymbol{\phi}$.

8.7 Polynomial Kernels

Many feature maps yield kernels that are computable much more cheaply than explicit feature vectors. For example,

$$K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^\top \mathbf{z})^2$$

corresponds to a quadratic feature map containing all products $x_i x_j$. Likewise,

$$K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^\top \mathbf{z} + c)^k$$

corresponds to all monomials up to degree k, with appropriate weighting.

Proposition 8.7.1. For fixed integer $k \geq 1$, the polynomial kernel

$$K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^\top \mathbf{z} + c)^k$$

corresponds to an implicit feature map containing monomials of degree up to k.

Interpretation

Kernalisation lets us work in a feature space of dimension on the order of d^k, while still computing the kernel in $O(d)$ time.

8.8 Gaussian Kernel

Another important example is the Gaussian kernel

$$K(\mathbf{x}, \mathbf{z}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{z}\|^2}{2\tau^2} \right).$$

This corresponds to an **infinite-dimensional** feature map.

Interpretation

The Gaussian kernel behaves like a similarity measure:

- close points have kernel value near 1,
- distant points have value near 0.

This is one reason it is so widely used.

8.9 Kernel Matrix and Valid Kernels

Suppose we choose any finite collection of points $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(n)}$.

Definition 8.9.1 (Kernel matrix)**.** The **kernel matrix** $K \in \mathbb{R}^{n \times n}$ is defined by

$$K = \begin{bmatrix} K(\mathbf{x}^{(1)}, \mathbf{x}^{(1)}) & \cdots & K(\mathbf{x}^{(1)}, \mathbf{x}^{(n)}) \\ \vdots & \ddots & \vdots \\ K(\mathbf{x}^{(n)}, \mathbf{x}^{(1)}) & \cdots & K(\mathbf{x}^{(n)}, \mathbf{x}^{(n)}) \end{bmatrix},$$

where the i, jth element $[K]_{ij} = K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$. If $K(\mathbf{x}, \mathbf{z}) = \langle \boldsymbol{\phi}(\mathbf{x}), \boldsymbol{\phi}(\mathbf{z}) \rangle$, then the kernel matrix must be symmetric and positive semidefinite.

Proposition 8.9.2. If K is induced by some feature map $\boldsymbol{\phi}$, then for every finite sample set, the kernel matrix is symmetric positive semidefinite.

Proof. Symmetry is immediate:

$$[K]_{ij} = \langle \boldsymbol{\phi}(\mathbf{x}^{(i)}), \boldsymbol{\phi}(\mathbf{x}^{(j)}) \rangle = \langle \boldsymbol{\phi}(\mathbf{x}^{(j)}), \boldsymbol{\phi}(\mathbf{x}^{(i)}) \rangle = [K]_{ji}.$$

Now let $\mathbf{z} \in \mathbb{R}^n$. Then

$$\mathbf{z}^\top K \mathbf{z} = \sum_{i=1}^{n} \sum_{j=1}^{n} z_i [K]_{ij} z_j = \sum_{i=1}^{n} \sum_{j=1}^{n} z_i \langle \boldsymbol{\phi}(\mathbf{x}^{(i)}), \boldsymbol{\phi}(\mathbf{x}^{(j)}) \rangle z_j.$$

Writing $\boldsymbol{\phi}(\mathbf{x}) = \begin{bmatrix} \phi_1(\mathbf{x}) & \phi_2(\mathbf{x}) & \cdots \end{bmatrix}^\top$, we get

$$\mathbf{z}^\top K \mathbf{z} = \sum_k \left(\sum_{i=1}^{n} z_i \phi_k(\mathbf{x}^{(i)}) \right)^2 \geq 0.$$

Hence K is positive semidefinite.

8.10 Mercer Condition

Theorem 8.10.1 (Mercer charactrisation, finite-sample form)**.** A function $K : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ is a valid kernel if and only if for every finite set $\{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(n)}\}$, the corresponding kernel matrix is symmetric positive semidefinite.

Interpretation

To verify that a candidate function is a valid kernel, it is enough to check that every kernel matrix it induces is symmetric PSD. This lets us define kernels without ever explicitly constructing the feature map.

9 Support Vector Machines

9.1 Binary Linear Classification Setup

For SVMs, labels are taken to be $y^{(i)} \in \{-1, 1\}$. The classifier is parametrised by $(\mathbf{w}, b)$, and predicts by the sign of the affine score:

$$h_{\mathbf{w},b}(\mathbf{x}) = \text{sign}(\mathbf{w}^\top \mathbf{x} + b).$$

The set $\{\mathbf{x} : \mathbf{w}^\top \mathbf{x} + b = 0\}$ is called the **separating hyperplane**.

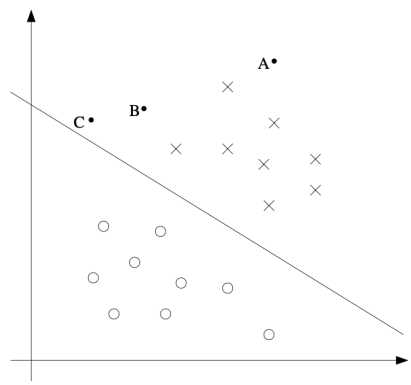


Figure 4: x's denote positive training examples, o's denote the negative training examples, a decision boundary (this is the line given by the equation $\mathbf{w}^\top \mathbf{x} + b = 0$, and is also called the **separating hyperplane**) is also shown.

9.2 Function Margin

Definition 9.2.1 (Functional margin of one example)**.** Given $(\mathbf{x}^{(i)}, y^{(i)})$, the functional margin of $(\mathbf{w}, b)$ with respect to this example is

$$\hat{\gamma}^{(i)} = y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b).$$

This is positive exactly when the example is correctly classified. If it is large, the classifier is not only correct but "confident" in sign.

Definition 9.2.2 (Functional margin of dataset)**.** For training set S,

$$\hat{\gamma} = \min\{\hat{\gamma}^{(i)}\}_{i=1}^{n}.$$

9.3 Geometric Margin

Functional margin is not scale-invariant: replacing $(\mathbf{w}, b)$ by $(c\mathbf{w}, cb)$ multiplies it by c, even though the classifier does not change. To fix this, define geometric margin.

Definition 9.3.1 (Geometric margin of one example)**.** The geometric margin is

$$\gamma^{(i)} = y^{(i)}\left[\left(\frac{\mathbf{w}}{\|\mathbf{w}\|}^{\top}\mathbf{x}^{(i)} + \frac{b}{\|\mathbf{w}\|}\right)\right] = \frac{y^{(i)}(\mathbf{w}^{\top}\mathbf{x}^{(i)} + b)}{\|\mathbf{w}\|}.$$

Definition 9.3.2 (Geometric margin of dataset)**.**

$$\gamma = \min\{\gamma^{(i)}\}_{i=1}^{n}.$$

9.4 Geometric meaning of Margin

Proposition 9.4.1. For a correctly classified point $(\mathbf{x}^{(i)}, y^{(i)})$, the geometric margin is precisely the signed Euclidean distance from $\mathbf{x}^{(i)}$ to the separating hyperplane.

Proof. Let $\mathbf{u} = \frac{\mathbf{w}}{\|\mathbf{w}\|}$, so $\mathbf{u}$ is the unit normal vector to the hyperplane. The orthogonal projection of $\mathbf{x}^{(i)}$ onto the hyperplane lies at

$$\mathbf{x}^{(i)} - \gamma^{(i)}\mathbf{u}.$$

Because this projected point lies on the hyperplane,

$$\mathbf{w}^{\top}\left(\mathbf{x}^{(i)} - \gamma^{(i)}\mathbf{u}\right) + b = 0.$$

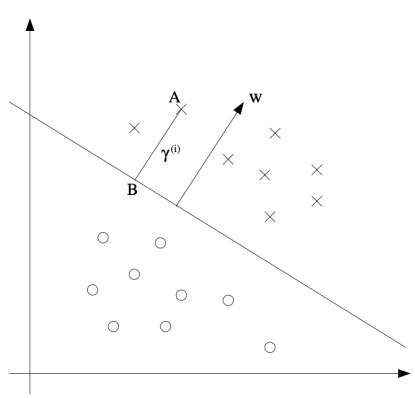
Substitute $\mathbf{u} = \frac{\mathbf{w}}{\|\mathbf{w}\|}$:

$$\mathbf{w}^{\top}\mathbf{x}^{(i)} - \gamma^{(i)}\|\mathbf{w}\| + b = 0.$$

Thus,

$$\gamma^{(i)} = \frac{\mathbf{w}^{\top}\mathbf{x}^{(i)} + b}{\|\mathbf{w}\|}.$$

Multiplying by $y^{(i)}$ gives the class-signed version.



9.5 Hard-Margin SVM

Suppose the data is linearly separable. We want the separating hyperplane with maximum geometric margin.

Theorem 9.5.1 (Hard-margin primal problem)**.** Maximusing the geometric margin is equivalent to solving

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1, \quad i = 1, \ldots, n.$$

Proof sketch. If we scale $(\mathbf{w}, b)$ so that the smallest functional margin equals 1, i.e.

$$\min\{y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b)\}_{i=1}^n = 1,$$

then the geometric margin becomes

$$\gamma = \frac{1}{\|\mathbf{w}\|}.$$

Hence maximising γ is equivalent to minimising $\|\mathbf{w}\|$, or equivalently $\frac{1}{2}\|\mathbf{w}\|^2$.

9.6 Lagrangian and Dual Problem

For the hard-margin primal problem, define the constraints

$$g_i(\mathbf{w}, b) = -y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) + 1 \leq 0.$$

The Lagrangian is

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i [y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) - 1], \quad \alpha_i \geq 0.$$

Differentiating with respect to $\mathbf{w}$ and b,

$$\nabla_{\mathbf{w}} \mathcal{L} = \mathbf{w} - \sum_{i=1}^n \alpha_i y^{(i)} \mathbf{x}^{(i)} = 0,$$

so

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y^{(i)} \mathbf{x}^{(i)},$$

and

$$\frac{\partial \mathcal{L}}{\partial b} = -\sum_{i=1}^n \alpha_i y^{(i)} = 0.$$

Substituting back yields the dual problem.

Theorem 9.6.1 (Dual form of hard-margin SVM)**.** The hard-margin SVM dual is

$$\begin{aligned}
&\max_{\boldsymbol{\alpha}} W(\boldsymbol{\alpha}) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y^{(i)} y^{(j)} \langle \mathbf{x}^{(i)}, \mathbf{x}^{(j)} \rangle \\
&\text{s.t. } \alpha_i \geq 0, \quad i = 1, \ldots, n \\
&\qquad \sum_{i=1}^{n} \alpha_i y^{(i)} = 0.
\end{aligned}$$

9.7 Support Vectors

From the KKT complementarity condition,

$$\alpha_i [y^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} + b) - 1] = 0.$$

Hence if $\alpha_i > 0$, then

$$y^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} + b) = 1.$$

So only points lying exactly on the margin boundaries have nonzero multipliers.

Definition 9.7.1 (Support vectors)**.** Training examples with $\alpha_i > 0$ are called **support vectors**.

Only a small subset of examples are support vectors, and only these determine the classifier.

Prediction formula

Using

$$\mathbf{w} = \sum_{i=1}^{n} \alpha_i y^{(i)} \mathbf{x}^{(i)},$$

we obtain

$$\mathbf{w}^\top \mathbf{x} + b = \sum_{i=1}^{n} \alpha_i y^{(i)} \langle \mathbf{x}^{(i)}, \mathbf{x} \rangle + b.$$

Thus prediction depends only on inner products with support vectors.

9.8 Why Kernelisation works for SVM

The dual objective and prediction formula involve the training data only through inner products $\langle \mathbf{x}^{(i)}, \mathbf{x}^{(j)} \rangle$. Therefore, we may replace these by a kernel

$$K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \langle \boldsymbol{\phi}(\mathbf{x}^{(i)}), \boldsymbol{\phi}(\mathbf{x}^{(j)}) \rangle,$$

and obtain the same optimisation problem in feature space, without explicitly computing $\boldsymbol{\phi}$. This is exactly the **kernel trick**.

Kernelised dual

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y^{(i)} y^{(j)} K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$$

subject to the same constraints.

9.9 Soft-Margin SVM

The hard-margin assumption fails when data is not linearly separable or contains outliers. To address this issue, we introduce slack variables ξ_i and the soft-margin objective.

Definition 9.9.1 (Soft-margin primal problem)**.**

$$\begin{aligned} \min_{\mathbf{w}, b, \boldsymbol{\xi}} \ & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^{n} \xi_i \\ \text{s.t. } & y^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1 - \xi_i, \quad i = 1, \ldots, n \\ & \xi_i \geq 0, \quad i = 1, \ldots, n. \end{aligned}$$

Here $C > 0$ controls the tradeoff between:

- large margin,
- low training classification error.

Interpretation

- Large C: strongly penalise violations, more emphasis on fitting training data;
- small C: tolerate violations more, emphasis larger margin and robustness.

9.10 Dual of Soft-Margin SVM

Soft-margin dual differs from the hard-margin dual only in the box constraint on α_i.

Theorem 9.10.1 (Dual from of soft-margin SVM)**.** The soft-margin dual is

$$\begin{aligned} \max_{\boldsymbol{\alpha}} \ & W(\boldsymbol{\alpha}) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y^{(i)} y^{(j)} \langle \mathbf{x}^{(i)}, \mathbf{x}^{(j)} \rangle \\ \text{s.t. } & 0 \leq \alpha_i \leq C, \quad i = 1, \ldots, n, \\ & \sum_{i=1}^{n} \alpha_i y^{(i)} = 0. \end{aligned}$$

KKT interpretation

KKT cases:

$$\begin{aligned} \alpha_i = 0 &\implies y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1, \\ 0 < \alpha_i < C &\implies y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) = 1, \\ \alpha_i = C &\implies y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \leq 1. \end{aligned}$$

These conditions distinguish:

- non-support vectors,
- margin support vectors,
- margin-violating / error points.

9.11 Coordinate Ascent and SMO

For a generic unconstrained maximisation

$$\max_{\boldsymbol{\alpha}} W(\alpha_1, \ldots, \alpha_n),$$

coordinate ascent updates

$$\alpha_i \leftarrow \operatorname*{argmax}_{\hat{\alpha}_i} W(\alpha_1, \ldots, \alpha_{i-1}, \hat{\alpha}_i, \alpha_{i+1}, \ldots, \alpha_n).$$

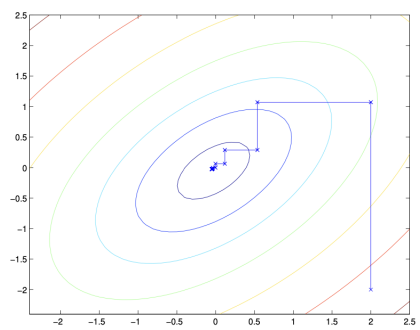


Figure 5: Illustration of the path of coordinate ascent on a quadratic function.

However, the SVM dual includes the equality constraint

$$\sum_{i=1}^{n} \alpha_i y^{(i)} = 0,$$

so updating a single coordinate alone cannot maintain feasibility. That is why SMO was motivated by updating **two** coordinates at a time.

Algorithm 8 Sequential Minimal Optimisation

Require: training data $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{n}$, kernel K, penalty C

Initialise $\boldsymbol{\alpha} \leftarrow \mathbf{0}$, choose b

while parameters not converged **do**

Select a pair (i, j) of multipliers to update

Hold all other α_k fixed

Optimise the dual objective with respect to α_i and α_j subject to:

$$\begin{aligned} \alpha_i &\geq 0, \\ \alpha_j &\leq C, \\ \alpha_i y^{(i)} + \alpha_j y^{(j)} &= \text{constant} \end{aligned}$$

Clip the updated value to satisfy box constraints

Update b

Check KKT conditions for convergence

end while

Why SMO is efficient

When all but two coordinates are fixed, the equality constraint reduces the feasible set to a line segment, and the objective becomes a quadratic in one variable. Hence each subproblem can be solved analytically up to clipping.

9.12 Geometric and Statistical Interpretation of SVM

Geometric view

SVM chooses the separating hyperplane with largest margin, which tends to be more robust perturbations than an arbitrary separating hyperplane.

Statistical view

Soft-margin SVM trades empirical fit against model complexity:

- $\frac{1}{2}\|\mathbf{w}\|^2$ controls complexity,
- $\sum_i \xi_i$ controls classification violations.

So SVM can be also seen as a regularised empirical risk minimisation problem.

9.13 Comparison with Logistic Regression

Logistic regression

- Models $P(y = 1|\mathbf{x})$,
- gives probabilistic outputs,
- minimises log-loss.

SVM

- Does not model probabilities directly,
- focuses on margin maximisation,
- yields a sparse representation through support vectors.

Practical difference

SVM is often a very strong off-the-shelf classifier, especially with kernels, while logistic regression is more interpretable probabilistically.

10 Generalisation

10.1 Overview

In supervised learning, minimising training loss is not the final goal. What we really care about it whether the learned model performs well on **unseen data**. This is the problem of **generalisation**. In this section we emphasise this distinction explicitly: training loss is computed on seen examples, while test loss is defined as expected loss on unseen samples drawn from the test distribution.

10.1.1 Main questions

1. Why can training error and test error differ?
2. What causes **underfitting** and **overfitting**?
3. How do **bias** and **variance** explain this?
4. How does **regularisation** help control model complexity?
5. How do we choose hyperparameters and models in practice?

10.2 Training Error vs Test Error

Let the training set be

$$\mathcal{D}_{\text{train}} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{n}.$$

Suppose a model $h_{\boldsymbol{\theta}}$ is trained by minimising a training objective such as MSE. Then the **training loss** is the empirical loss on the training set.

Definition 10.2.1 (Training loss)**.** For regression with squared loss, the training loss is

$$\hat{J}(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n}\left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)}\right)^2.$$

Definition 10.2.2 (Test loss)**.** If $(\mathbf{x}, y) \sim \mathcal{D}$ is a fresh unseen example from the test distribution, then the test loss is

$$L(\boldsymbol{\theta}) = E_{(\mathbf{x},y)\sim\mathcal{D}}[(h_{\boldsymbol{\theta}}(\mathbf{x}) - y)^2],$$

which is the population-level expectation over unseen data.

Key point

A small training loss does **not** automatically imply a small test loss. The difference between the two is often called the **generalisation gap**.

10.3 Underfitting and Overfitting

Definition 10.3.1 (Underfitting)**.** A model **underfits** if it performs poorly even on the training data. This usually means the model class if too simple, or the hypothesis family cannot represent the true pattern in the data.

Definition 10.3.2 (Overfitting)**.** A model **overfits** if it fits the training data very well but performs poorly on unseen test examples. This usually means the model is too flexible, it captures noise or spurious patterns specific to the finite training set.

Intuition

- Underfitting - model is too weak to learn the real signal.
- Overfitting - model is too strong and starts fitting noise.

10.4 Bias and Variance: Informal View

Informal bias

Bias measures how far the model family is, in principle, from the ground truth. If even with infinite data, the best model in the class would still be wrong, then the model family has high bias.

Informal variance

Variance measures how sensitive the learned model is to the particular sampled training dataset. If small changes in the dataset lead to very different learned models, then the learning procedure has high variance.

Typical pattern

- Simple models: high bias, low variance.
- Complex models: low bias, high variance.

10.5 Bias-Variance Decomposition for Regression

Assume the data is generated as

$$y = h^*(\mathbf{x}) + \xi, \quad \xi \sim \mathcal{N}(0, \sigma^2),$$

where $h^*(\mathbf{x})$ is the true target function, ξ is observation noise.

Let $\hat{h}_S$ denote the model learned from training set S. Fix an input $\mathbf{x}$. Define

$$\mathrm{MSE}(\mathbf{x}) = E_{S,\xi}[(\hat{h}_S(\mathbf{x}) - y)^2].$$

Also define the **average learned predictor**

$$h_{\text{avg}}(\mathbf{x}) = E_S[\hat{h}_S(\mathbf{x})].$$

Then

$$\text{MSE}(\mathbf{x}) = \underbrace{\sigma^2}_{\text{unavoidable noise}} + \underbrace{\Big(h_{\text{avg}}(\mathbf{x}) - h^*(\mathbf{x})\Big)^2}_{\text{bias}^2} + \underbrace{\text{Var}(\hat{h}_S(\mathbf{x}))}_{\text{variance}}.$$

Interpretation

- **Noise term**: cannot be removed. **Bias term**: error due to model family being systematically wrong;
- **variance term**: error due to sensitivity to the sampled dataset.

Important note

Bias-variance decomposition for classification is much less clean, and there is no universally agreed formalism.

10.6 Bias-Variance Tradeoff

Proposition 10.6.1. Increasing model complexity often decreases bias but increases variance.

Consequence

Test error often behaves like

$$\text{test error} \approx \text{bias}^2 + \text{variance} + \text{noise},$$

and therefore may first decrease and then increase as model complexity grows.

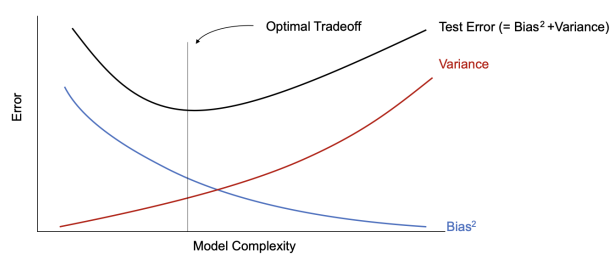


Figure 6: An illustration of the typical bias-variance tradeoff.

Practical interpretation

- A linear model may underfit a quadratic truth;
- a high-degree polynomial may overfit a small noisy dataset;
- an intermediate model often gives the best tradeoff.

10.7 Double Descent

The classical U-shaped curve for bias-variance tradeoff is not universal. In modern machine learning, especially in overparameterised models, test error can exhibit **doulble descent**.

Definition 10.7.1 (Model-wise double descent)**.** As model size increases:

1. test error first decreases,
2. then increases near the interpolation threshold,
3. then decreases again in the overparameterised regime.

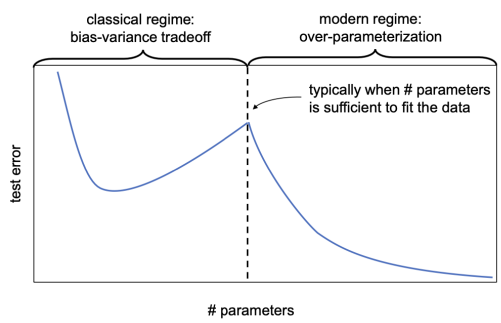


Figure 7: A typical model-wise double descent phenomenon. As the number of parameters increases, the test error first decreases when the number of parameters is smaller than the training data. Then in the overparameterised regime, the test error decreases again.

Definition 10.7.2 (Sample-wise double descent)**.** As the number of training examples increases, test error may:

1. decrease,
2. then peak around $n \approx d$,
3. then decease again.

Interpretation

The classical bias-variance tradeoff remains useful, but modern high-dimensional settings are more complicated:

- the number of parameters is not always the right complexity measure;
- optimisation and implicit regularisation matter;
- very large overparameterised models can still generalise well.

Takeaway

Double descent says that "larger model" does not always mean "worse generalisation after some point"; in modern ML, test error may decrease again beyond the interpolation threshold.

11 Regularisation

11.1 Overview

When a model is too flexible, we often control its complexity by adding a penalty term to the training objective.

Definition 11.1.1 (Regularised objective)**.** Let $J(\boldsymbol{\theta})$ be the original training loss. A regularised objective is

$$J_\lambda(\boldsymbol{\theta}) = J(\boldsymbol{\theta}) + \lambda R(\boldsymbol{\theta}),$$

where $R(\boldsymbol{\theta}) \geq 0$ is the regulariser, and $\lambda \geq 0$ is the regularisation strength.

Interpretation

Regularisation tries to find a model that fits the data well, while remaining "simple" according to $R(\boldsymbol{\theta})$.

Effect of λ

- $\lambda = 0$: no regularisation;
- small λ: mostly fit data, lightly penalise complexity;
- large λ: strong simplicity bias, may cause underfitting.

11.2 ℓ_2 Regularisation

Definition 11.2.1 (ℓ_2 regularisation)**.** The most common regulariser is

$$R(\boldsymbol{\theta}) = \frac{1}{2}\|\boldsymbol{\theta}\|_2^2.$$

Then the regularised objective becomes

$$J_\lambda(\boldsymbol{\theta}) = J(\boldsymbol{\theta}) + \frac{\lambda}{2}\|\boldsymbol{\theta}\|_2^2.$$

It is also known as the **weight decay** in deep learning. If gradient descent uses learning rate η, then one step on the regularised loss gives

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla J_\lambda(\boldsymbol{\theta}) = (1 - \eta\lambda)\boldsymbol{\theta} - \eta \nabla J(\boldsymbol{\theta}).$$

So the weights are multiplicatively shrunk before the usual gradient step.

ℓ_2 regularisation prefers smaller parameter norms, which often leads to smoother and less overfit models.

11.3 ℓ_1 Regularisation and Sparsity

Definition 11.3.1 (L_1 regularisation)**.** Another common regulariser is

$$R(\boldsymbol{\theta}) = \|\boldsymbol{\theta}\|_1.$$

This is closely related to LASSO.

ℓ_1 regularisation encourages **sparsity**, meaning many coordinates of $\boldsymbol{\theta}$ become zero or close to zero. It is useful when we believe only a few input dimensions matter.

11.4 Regularisation beyond Linear Models

Regularisation is broader than just ℓ_1 or ℓ_2. In deep learning, common regularisers include:

- weight decay,
- dropout,
- data augmentation,
- spectral norm regularisation,
- Lipschitz regularisation.

11.5 Implicit Regularisation

Modern optimisation algorithms may regularise the learned solution even when the explicit loss function does not contain a regulariser.

Definition 11.5.1 (Implicit regularisation)**.** Implicit regularisation refers to the phenomenon that the optimiser itself biases learning toward certain types of solutions.

In deep learning there may be many global minima with similar training loss but very different generalisation performance. Different optimisers, initialisations, learning-rate schedules, or batch sizes may converge to different such minima.

Key idea

Even when two models achieve the same training loss, or even zero training error, they may generalise differently because the optimisation path selects different solutions. This is illustrated in Figure 8.

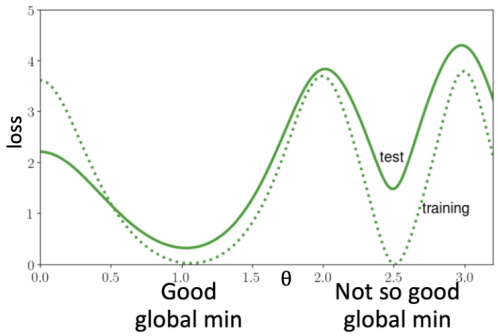


Figure 8: An Illustration that different global minima of the training loss can have different test performance.

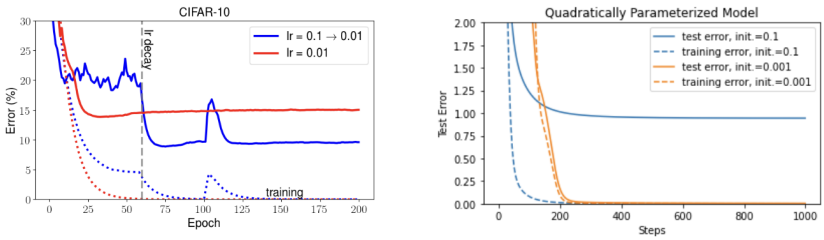


Figure 9: **Left**: Performance of neural networks trained by two different learning rate schedules on the CIFAR-10 dataset. Although both experiments used exactly same regularised losses and optimisers fit the training data perfectly, the generalisation performances differ much. **Right**: On a different synthetic dataset, optimisers with different initialisations have the same training error but different generalisation performance.

Practical heuristics

In many settings, the following often help guide optimisation toward more generalisable solutions:

- larger initial learning rate,
- smaller initialisation,
- smaller batch size,
- momentum.

Takeaway

Optimiser choice affects not only training speed, but also which solution is found—and therefore how well the model generalises.

11.6 Model Selection and Cross Validation

Regularisation introduces hyperparameters such as λ. We also often need to choose:

- model class,
- polynomial degree,
- bandwidth,
- tree depth,
- number of neighbours,
- learning-rate schedule.

Main idea

We need a way to estimate generalisation performance using the available data. That is the purpose of **cross validation**.

11.7 Hold-Out Cross Validation

Algorithm 9 Hold-out cross validation

Require: dataset S, candidate models $M_1, \ldots, M_d$
  Randomly split S into S_{train} (e.g. 70%), S_{cv} (e.g. 30%)
  for each model M_i **do**
    train M_i on S_{train}
    Evaluate validation error on S_{cv}
  end for
  Select the model with the smallest validation error
  Optionally retrain the selected model on the full dataset S

Advantage

Simple and easy to implement.

Limitation

It "wastes" part of the dataset for validation, which can be problematic when data is scarce.

11.8 k-Fold Cross Validation

Definition 11.8.1 (k-fold cross validation)**.** Split the dataset into k disjoint subsets $S_1, \ldots, S_k$. For each candidate model:

- train on $k-1$ folds,
- validate on the remaining fold,
- repeat for all k choices of hold-out fold,
- average the validation error.

Algorithm 10 k-fold cross validation

Require: dataset S, candidate models $M_1, \ldots, M_d$, number of folds k

```
Split S into disjoint folds S_1, ..., S_k
for each model M_i do
    for j ∈ {1, ..., k} do
        Train M_i on S \ S_j
        Evaluate error on S_j
    end for
    Compute average validation error over j
end for
Choose the model with smallest average validation error
Retrain the chosen model on the full dataset S
```

It is recommended to choose $k = 10$ as a common practical choice.

Special case: Leave-one-out cross validation

When $k = n$, we train on all but one example each time. This is called **leave-one-out cross validation**.

Tradeoff

- larger k: uses data more efficiently,
- but is more computationally expensive.

11.9 Bayesian View and MAP Estimation

We can connect regularisation to Bayesian statistics. In the frequentist view, $\boldsymbol{\theta}$ is a fixed but unknown parameter. In the Bayesian view, $\boldsymbol{\theta}$ is treated as a random variable with prior $p(\boldsymbol{\theta})$.

Given training data S, the posterior is

$$p(\boldsymbol{\theta}|S) = \frac{p(S|\boldsymbol{\theta})p(\boldsymbol{\theta})}{p(S)}.$$

Definition 11.9.1 (MAP estimate)**.** The maximum a posteriori estimate is

$$\boldsymbol{\theta}_{\text{MAP}} = \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \left[\prod_{i=1}^{n} p(y^{(i)}|\mathbf{x}^{(i)}, \boldsymbol{\theta}) \right] p(\boldsymbol{\theta}).$$

This differs from MLE only by the extra prior term.

11.10 MAP and Regularisation

A particularly important case is a Gaussian prior:

$$\boldsymbol{\theta} \sim \mathcal{N}(\mathbf{0}, \tau^2 I).$$

Then maximising the posterior is equivalent to minimising a loss with an ℓ_2 penalty.

Proposition 11.10.1. A Gaussian prior on $\boldsymbol{\theta}$ leads to ℓ_2 regularisation in MAP estimation. This is exactly why ℓ_2 regularisation has a clean Bayesian interpretation.

Practical meaning

Regularisation is not just an optimisation trick. It can be interpreted as encoding prior beliefs about what kinds of parameters are plausible.

11.11 Big Picture

We can now summarise the conceptual flow:

Generalisation problem

Training error is not enough; we care about unseen data.

Bias-variance framework

Test error comes from:

- irreducible noise,
- bias,
- variance.

Regularisation

Adds a preference for simpler / structured solutions.

Cross validation

Provides a practical way to choose model complexity and hyperparameters.

Bayesian perspective

Interprets regularisation as prior information.

Implicit regularisation

Explains why optimisation details matter, especially in modern overparameterised models.

Unsupervised Learning

12 K-Means Clustering

12.1 Overview

In the clustering problem, we are given data points

$$\{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(n)}\}, \quad \mathbf{x}^{(i)} \in \mathbb{R}^d,$$

but no labels $y^{(i)}$. The goal is to partition the data into a small number of cohesive groups called **clusters**. This is therefore an **unsupervised learning** problem. Among the simplest and most widely used clustering methods is K-means.

Main idea

K-means seeks K representative points, called **centroids**, and assigns each data point to the nearest centroid. Then each centroid is updated to the mean of the points assigned to it. This alternating procedure is repeated until convergence.

When to use

K-means is most appropriate when:

- clusters are approximately spherical in Euclidean space;
- cluster structure is primarily based on distance to a centre;
- a fast an interpretable clustering baseline is desired.

12.2 Problem Formulation

Suppose we want to partition the dataset into K clusters.

Definition 12.2.1 (Cluster assignment variables)**.** For each example $\mathbf{x}^{(i)}$, define its cluster assignment

$$c^{(i)} \in \{1, 2, \ldots, K\}.$$

Thus $c^{(i)} = j$ means that $\mathbf{x}^{(i)}$ is assigned to cluster j.

Definition 12.2.2 (Centroids)**.** Let the centroid of cluster j by

$$\boldsymbol{\mu}_j \in \mathbb{R}^d, \quad j = 1, \ldots, K.$$

The pair $(\mathbf{c}, \boldsymbol{\mu})$ with $\mathbf{c} = (c^{(1)}, \ldots, c^{(n)})$ and $\boldsymbol{\mu} = (\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K)$ completely specifies a K-means clustering.

12.3 The K-Means Algorithm

Initialise centroids randomly, then repeatedly assign each point to the nearest centroid and move each centroid to the mean of the points assigned to it.

Algorithm 11 K-Means

Require: data points $\{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(n)}\} \subset \mathbb{R}^d$, number of clusters K

Initialise centroids $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K$ randomly

while not converged **do**

for $i \in \{1, \ldots, n\}$ **do**

$c^{(i)} \leftarrow \operatorname{argmin}_j \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_j\|^2$

end for

for $j \in \{1, \ldots, K\}$ **do**

$$\boldsymbol{\mu}_j \leftarrow \frac{\sum_{i=1}^n \mathbf{1}_{\{c^{(i)}=j\}} \mathbf{x}^{(i)}}{\sum_{i=1}^n \mathbf{1}_{\{c^{(i)}=j\}}}$$

end for

end while

Interpretation

- **Assignment step**: fix centroids, choose the best cluster for each point.
- **Update step**: fix assignments, choose the best centroid for each cluster.

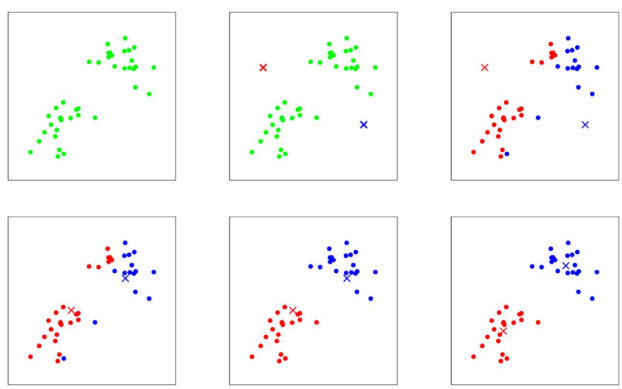


Figure 10: (a) Original dataset. (b) Random initial cluster centroids (in this instance, not chosen to be equal to two training examples). (c-f) Illustration of running two iterations of K-means. In each iteration, we assign each training example to the closest cluster centroid (shown by "painting" the training examples the same colour as the cluster centroid to which is assigned); then we move each cluster centroid to the mean of the points assigned to it.

12.4 Distortion Function

To understand what K-means is optimising, define the **distortion function**

$$J(\mathbf{c}, \boldsymbol{\mu}) = \sum_{i=1}^n \left\|\mathbf{x}^{(i)} - \boldsymbol{\mu}_{c^{(i)}}\right\|^2.$$

This is the sum of squared distances from each data point to the centroid of the cluster to which it is assigned.

Definition 12.4.1 (Distortion / within-cluster sum of squares)**.** The K-means objective js

$$J(\mathbf{c}, \boldsymbol{\mu}) = \sum_{i=1}^{n} \left\| \mathbf{x}^{(i)} - \boldsymbol{\mu}_{c^{(i)}} \right\|^2.$$

Interpretation

K-means seeks a clustering that makes points close to their assigned centroids. Thus it tries to produce compact, low-variance clusters.

12.5 K-means as Coordinate Descent

K-means is exactly **coordinate descent** on $J(\mathbf{c}, \boldsymbol{\mu})$: first minimise over assignments $\mathbf{c}$ with centroids fixed, then minimise over centroids $\boldsymbol{\mu}$ with assignments fixed.

Proposition 12.5.1. For fixed centroids $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K$, the assignment step

$$c^{(i)} := \operatorname*{argmin}_j \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_j\|^2$$

minimises $J(\mathbf{c}, \boldsymbol{\mu})$ with respect to $\mathbf{c}$.

Proof. If $\boldsymbol{\mu}$ is fixed, then

$$J(\mathbf{c}, \boldsymbol{\mu}) = \sum_{i=1}^{n} \left\| \mathbf{x}^{(i)} - \boldsymbol{\mu}_{c^{(i)}} \right\|^2.$$

Each term depends only on $c^{(i)}$, so the minimisation over $\mathbf{c}$ decomposes into n separate minimisations:

$$c^{(i)} \in \operatorname*{argmin}_{j \in \{1, \ldots, K\}} \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_j\|^2.$$

Thus choosing the nearest centroid for each point minimises the objective.

Proposition 12.5.2. For fixed assignments $\mathbf{c}$, the update step

$$\boldsymbol{\mu}_j = \frac{\sum_{i=1}^{n} \mathbf{1}_{\{c^{(i)}=j\}} \mathbf{x}^{(i)}}{\sum_{i=1}^{n} \mathbf{1}_{\{c^{(i)}=j\}}}$$

minimises $J(\mathbf{c}, \boldsymbol{\mu})$ with respect to $\boldsymbol{\mu}_j$.

Proof. Fix j, and let $S_j = \{i : c^{(i)} = j\}$ be the set of points assigned to cluster j. The part of the objective depending on $\boldsymbol{\mu}_j$ is

$$J_j(\boldsymbol{\mu}_j) = \sum_{i \in S_j} \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_j\|^2 = \sum_{i \in S_j} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_j)^\top (\mathbf{x}^{(i)} - \boldsymbol{\mu}_j).$$

Differentiating with respect to $\boldsymbol{\mu}_j$,

$$\nabla_{\boldsymbol{\mu}_j} J_j(\boldsymbol{\mu}_j) = -2 \sum_{i \in S_j} \mathbf{x}^{(i)} + 2|S_j|\boldsymbol{\mu}_j.$$

Setting the gradient to zero gives

$$|S_j|\boldsymbol{\mu}_j = \sum_{i \in S_j} \mathbf{x}^{(i)},$$

hence

$$\boldsymbol{\mu}_j = \frac{1}{|S_j|} \sum_{i \in S_j} \mathbf{x}^{(i)}.$$

Therefore the centroid that minimises within-cluster squared error is the cluster mean.

12.6 Initialisation and Multiple Restarts

Run K-means several times with different random initialisations, and choose the solution with the smallest distortion J.

Algorithm 12 K-means with multiple restarts

Require: data points $\{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(n)}\}$, number of clusters K, number of restarts R
  for $r \in \{1, \ldots, R\}$ **do**
    Randomly initialise centroids
    Run K-means until convergence
    Compute final distortion J_r
  end for
  return the clustering with the smallest distortion among $\{J_1, \ldots, J_R\}$

In modern implementations, K-means++ is often used for smarter initialisation.

12.7 Geometric Interpretation

K-means induces a partition of the input space into Voronoi-type regions:

$$R_j = \{\mathbf{x} : \forall \ell, \|\mathbf{x} - \boldsymbol{\mu}_j\|^2 \leq \|\mathbf{x} - \boldsymbol{\mu}_\ell\|^2\}.$$

Each point is assigned to the region of its nearest centroid.

Consequence

The cluster boundaries are piecewise linear bisectors between centroids.

Interpretation

K-means is therefore best suited to cluster shapes that can be reasonably represented by a centroid and Euclidean distance.

Limitation

It does not naturally model:

- elongated elliptical clusters,
- manifolds,
- clusters with very different densities or sizes,
- nonconvex cluster shapes.

This is one reason GMM and density-based methods are important later.

12.8 Choosing K

The number of clusters K is a hyperparameter and must be specified in advance.

Practical strategies

Common heuristic include:

- domain knowledge,
- elbow plots of distortion vs K,
- validation on downstream tasks,
- silhouette-style diagnostics.

Important caveat

The distortion J always decreases as K increases, so training distortion alone cannot determine the correct K. This is analogous to model complexity in supervised learning.

12.9 Empty Clusters

A practical issue is that some cluster may receive no points during an assignment step, making its centroid undefined.

Common fixes

- Reinitialise that centroid randomly,
- reinitialise it to a point with large current distortion;
- merge / remove empty clusters depending on implementation.

This is mostly an implementation issue.

12.10 Strengths and Limitations

Strengths

- Simple and fast;
- easy to implement;
- scales well to moderately large datasets;
- often provides a strong clustering baseline;
- objective and updates are mathematically transparent.

Limitations

- Requires choosing K in advance;
- sensitive to initialisation;
- converges only to a local optimum;
- assumes Euclidean, centroid-based cluster structure;
- sensitive to feature scaling.

12.11 Importance of Feature Scaling

Because K-means uses Euclidean distance, the relative scale of features matters greatly.

Practical rule

Standardise or normalise features before running K-means, unless the original scale is itself meaningful.

13 Density-based Clustering

K-Means works well when clusters are roughly convex and centroid-like, but it struggles with:

- curved or irregularly shaped clusters,
- clusters of unequal size,
- outliers,
- cases where "cluster centre" is not meaningful.

Density-based methods instead define a cluster as a region in which data points are packed densely together.

Informally, a cluster is a maximal connected high-density region. Points in sparse regions are treated as **noise** or **outliers**. The most classical density-based clustering algorithm is **DBSCAN**.

13.1 DBSCAN: core definitions

Definition 13.1.1 (ε-neighbourhood)**.** For a point $\mathbf{x} \in \mathbb{R}^d$, its **$\varepsilon$-neighbourhood** is

$$N_\varepsilon(\mathbf{x}) = \{\mathbf{z} \in \mathcal{D} : \mathrm{dist}(\mathbf{z}, \mathbf{x}) \leq \varepsilon\},$$

where dist is usually Euclidean distance, though other metrics may also be used.

Definition 13.1.2 (Core point)**.** Given parameters $\varepsilon > 0$ and $\mathrm{MinPts} \in \mathbb{N}$, a point $\mathbf{x}$ is a **core point** if

$$|N_\varepsilon(\mathbf{x})| \geq \mathrm{MinPts}\,.$$

So a core point lies in a sufficiently dense region.

Definition 13.1.3 (Border point)**.** A point $\mathbf{x}$ is a **border point** if:

1. $\mathbf{x}$ is **not** a core point, and
2. $\mathbf{x} \in N_\varepsilon(\mathbf{y})$ for some core point $\mathbf{y}$.

Thus a border point is near a dense region, but does not itself have enough nearby neighbours to be core.

Definition 13.1.4 (Noise point)**.** A point is a **noise point** (or outlier) if it is neither a core point nor a border point.

Definition 13.1.5 (Directly density-reachable)**.** A point $\mathbf{y}$ is **directly density-reachable** from $\mathbf{x}$ if:

1. $\mathbf{y} \in N_\varepsilon(\mathbf{x})$ and
2. $\mathbf{x}$ is a core point.

Note that this relation is generally **not symmetric**.

Definition 13.1.6 (Density-reachable)**.** A point $\mathbf{y}$ is **density-reachable** from $\mathbf{x}$ if there exists a sequence

$$\mathbf{x} = \mathbf{x}_1, \mathbf{x}_1, \ldots, \mathbf{x}_m = \mathbf{y}$$

such that $\mathbf{x}_{j+1}$ is directly density-reachable from $\mathbf{x}_j$ for each j. This allows us to move from one point to another through a chain of nearby core points.

Definition 13.1.7 (Density-connected)**.** Two points $\mathbf{x}$ and $\mathbf{y}$ are **density-connected** if there exists a point $\mathbf{z}$ such that both $\mathbf{x}$ and $\mathbf{y}$ are density-reachable from $\mathbf{z}$. A DBSCAN cluster can be viewed as a maximal density-connected set.

13.2 DBSCAN Algorithm

Algorithm 13 DBSCAN

```
Require: dataset D, radius ε, minimum number of neighbours MinPts
  Mark all points as unvisited
  cluster_id ← 0
  for each x ∈ D do
      if x is visited then
          continue
      end if
      Mark x as visited
      neighbours ← N_ε(x)
      if |neighbours| < MinPts then
          Mark x as noise
      else
          cluster_id ← cluster_id + 1
          expand_cluster(x, neighbours, cluster_id)
      end if
  end for
  return cluster labels and noise points
```

Algorithm 14 expand_cluster

Require: $\mathbf{x}$, neighbours, cluster_id

```
Assign x to cluster_id
for each y ∈ neighbours do
    if y is not visited then
        Mark y as visited
        neighbours_y ← N_ε(y)
        if |neighbours_y| ≥ MinPts then
            neighbours ← neighbours ∪ neighbours_y
        end if
    end if
    if y is not yet assigned to any cluster then
        Assign y to cluster_id
    end if
end for
```

Interpretation

The procedure is essentially:

1. pick a point,
2. check whether it lies in a dense region,
3. if yes, start a new cluster,
4. recursively absorb all points connected to that dense region.

So DBSCAN performs a kind of **graph expansion** over local dense neighbourhood.

A useful way to think about DBSCAN is make a graph whose vertices are data points, connect nearby points if distance $\leq \varepsilon$. And identify core points, clulsters are connected components formed by core points, together with their attached border points.

Proposition 13.2.1. DBSCAN can recover clusters with non-convex and irregular shapes, provided those clusters form connected dense regions under the chosen ε and MinPts.

Unlike K-means, DBSCAN does not fit each cluster by a centroid. Instead, if only requires local neighbourhood connectivity in dense regions. Therefore, if points lie along a curved manifold or elongated shape, the cluster can still be recovered through chains of local neighbours.

13.3 Advantages and Limitations of DBSCAN

Advantages

- No need to specify the number of clusters K beforehead.
- Can detect arbitrary shaped clusters.

- Naturally identifies outliers as noise.
- Often more meaningful than K-means when geometry is complex.

Limitations

- Sensitive to the choice of ε and MinPts.
- Difficult when clusters have **different densities**.
- In high dimensions, distance neighbourhoods become less informative.
- Performance may degrade if neighbourhood queries are expensive.

13.4 Choose ε and MinPts

A common heuristic is to set MinPts to something like $d + 1$, $2d$, or a small integer such as 4 to 10. Inspect the k-distance plot: for each point, compute distance to its kth nearest neighbour (often k=MinPts); sort these distances; look for an "elbow". The elbow roughly indicates a threshold separating dense regions from sparse noise.

13.5 Complexity

If neighbourhood queries are done naively, each query may cost $O(n)$, so total cost is roughly $O(n^2)$.

If a spatial index such as a **kd-tree** or **ball tree** is effective, neighbour search may be faster, so runtime can be closer to $O(n \log n)$ in favourable low-dimensional settings. In high dimensions, indexing often becomes much less effective.

14 Hierarchical Clustering

Instead of producing only one flat partition, hierarchical clustering creates a sequence of nested clusters:

- small groups merge into larger groups, or
- a large group splits into smaller groups.

This yields a **tree structure** called the **dendrogram**. So hierarchical clustering is useful when cluster structure may exist at multiple resolutions.

14.1 Two paradigms

Definition 14.1.1 (Agglomerative clustering)**. Agglomerative hierarchical clustering** is a bottom-up method:

- start with each point as its own cluster,
- repeatedly merge the two closest clusters,
- continue until only one cluster remains.

Definition 14.1.2 (Divisive clustering)**. Divisive hierarchical clustering** is a top-down method:

- start with all points in one cluster,
- repeatedly split clusters,
- continue until singleton clusters or some stopping rule is reached.

In practice, **agglomerative** clustering is much more common.

14.2 Agglomerative clustering framework

Suppose at some stage we have clusters

$$C_1, C_2, \ldots, C_m.$$

At each iteration:

1. compute the inter-cluster dissimilarity between every pair (C_a, C_b),
2. merge the closest pair,
3. update the distance structure.

The whole method depends crucially on **how we define distance between clusters**.

14.3 Linkage criteria

Definition 14.3.1 (Single linkage)**.** The distance between clusters A and B is

$$d_{\text{single}}(A, B) = \min\{\text{dist}(\mathbf{x}, \mathbf{y}) : \mathbf{x} \in A,\ \mathbf{y} \in B\}.$$

This merges clusters whose closest points are very near.

Definition 14.3.2 (Complete linkage)**.** The distance between clusters A and B is

$$d_{\text{complete}}(A, B) = \max\{\text{dist}(\mathbf{x}, \mathbf{y}) : \mathbf{x} \in A,\ \mathbf{y} \in B\}.$$

This is more conservative: two clusters merge only if all cross-cluster point pairs are relatively close.

Definition 14.3.3 (Average linkage)**.** The distance between clusters A and B is

$$d_{\text{average}}(A, B) = \frac{1}{|A||B|} \sum_{\mathbf{x} \in A} \sum_{\mathbf{y} \in B} \text{dist}(\mathbf{x}, \mathbf{y}).$$

This averages all pairwise distances across the two clusters.

Definition 14.3.4 (Centroid linkage)**.** The distance between clusters A and B is

$$d_{\text{centroid}}(A, B) = \|\boldsymbol{\mu}_A - \boldsymbol{\mu}_B\|_2$$

where

$$\boldsymbol{\mu}_A = \frac{1}{|A|} \sum_{\mathbf{x} \in A} \mathbf{x}, \quad \boldsymbol{\mu}_B = \frac{1}{|B|} \sum_{\mathbf{y} \in B} \mathbf{y}.$$

This resembles the centroid idea from K-means.

Definition 14.3.5 (Ward linkage)**.** Ward's method merges the pair of clusters that yields the smallest increase in within-cluster sum of squares. Equivalently, if A and B are merged, the merge cost is

$$\Delta(A, B) = \sum_{\mathbf{x} \in A \cup B} \|\mathbf{x} - \boldsymbol{\mu}_{A \cup B}\|_2^2 - \sum_{\mathbf{x} \in A} \|\mathbf{x} - \boldsymbol{\mu}_A\|_2^2 - \sum_{\mathbf{x} \in B} \|\mathbf{x} - \boldsymbol{\mu}_B\|_2^2.$$

Ward linkage tends to prefer compact, spherical clusters.

14.4 Agglomerative clustering algorithm

Algorithm 15 AgglomerativeClustering

Require: dataset $\mathcal{D}$, linkage rule

```
Initialise each point as a singleton cluster
Compute pairwise cluster distances
while more than one cluster remains do
    Choose clusters A, B with smallest linkage distance
    Merge A and B into a new cluster C
    Remove A and B from the active cluster set
    Add C to the active cluster set
    Update distances from C to all remaining clusters
end while
return dendrogram of all merges
```

14.5 Properties of different linkage rules

Proposition 14.5.1 (Single linkage favours chaining)**.** Single linkage is sensitive to "chains" of nearby points: clusters may grow by thin bridge of points even when the overall groups are not compact.

Since it only uses the minimum pairwise distance,

$$d_{\text{single}}(A, B) = \min\{\text{dist}(\mathbf{x}, \mathbf{y}) : \mathbf{x} \in A,\ \mathbf{y} \in B\},$$

one very close pair can force a merge. Hence long, string-like clusters are often formed.

Proposition 14.5.2 (Complete linkage favours compactness)**.** Complete linkage tends to produce more compact clusters than single linkage.

Because it uses the maximum cross-cluster distance,

$$d_{\text{complete}}(A, B) = \max\{\text{dist}(\mathbf{x}, \mathbf{y}) : \mathbf{x} \in A,\ \mathbf{y} \in B\}$$

a merge occurs only when the entire combined cluster remains reasonably tight.

Proposition 14.5.3 (Ward linkage is variance-minimising)**.** Ward linkage is the hierarchical analogue most closely related to K-means-style compactness objective.

Ward's merge criterion explicitly minimises the increase in within-cluster sum of squares. Therefore it strongly favours spherical and low-variance clusters.

14.6 Output: obtaining a flat clustering

Hierarchical clustering itself gives a dendrogram, not necessarily a final partition. To get k clusters, we may:

- cut the tree at a fixed height, or
- cut it so that exactly k connected components remain.

This flexibility is one of the major practical benefits of hierarchical methods.

14.7 Complexity

For agglomerative clustering, a naive implementation can be expensive, in which storing pairwise distances already costs $O(n^2)$, repeated merging and updating may cost $O(n^3)$ naively.

With specialised algorithms, some linkage rules can be implemented in $O(n^2)$ time, but hierarchical clustering is still usually more expensive than K-means on large datasets.

14.8 Advantages and Limitations

Advantages

- No need to specify K upfront.
- Produces a full hierarchy, not just one partition.
- Dendrogram is highly interpretable.
- Works with many distance / similarity notions.

Limitations

- Computationally expensive for large n.
- Greedy merges are irreversible.
- Sensitive to scaling and distance choice.
- Different linkage rules can produce very different results.

15 Principal Components Analysis

15.1 Overview

In many datasets, the ambient dimension d is much larger than the true intrinsic dimension of the data. The data may lie approximately near a lower-dimensional affine subspace, and much of the variation may be explained by only a few directions. **Principal Component Analysis** (PCA) is a classical method for finding such directions.

PCA is one of the most important techniques in unsupervised learning because it gives:

- a method for **dimensionality reduction**;
- a method for **data compression**;
- a way to identify the dominant directions of variation in the data.

Given centred data points $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(n)} \in \mathbb{R}^d$, PCA finds orthonormal directions $\mathbf{u}_1, \mathbf{u}_2, \ldots, \mathbf{u}_k$ such that projecting the data onto the span of these directions preserves as much variance as possible, or equivalently minimises reconstruction error.

15.2 Data Centering

PCA should be applied to **centred data**.

Definition 15.2.1 (Sample mean)**.** Given data points $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(n)}$, the sample mean is

$$\bar{\mathbf{x}} = \frac{1}{n} \sum_{i=1}^{n} \mathbf{x}^{(i)}.$$

Definition 15.2.2 (Centred data)**.** The centred version of each point is

$$\tilde{\mathbf{x}}^{(i)} = \mathbf{x}^{(i)} - \bar{\mathbf{x}}.$$

Unless otherwise stated, PCA is performed on the centred dataset

$$\tilde{\mathbf{x}}^{(1)}, \ldots, \tilde{\mathbf{x}}^{(n)}.$$

Centring is important, without it, the first principal component may point toward the mean of the data rather than toward the direction of maximal variation.

15.3 First Principal Component: Variance Maximisation View

Suppose we want to find a unit vector $\mathbf{u}$ ontp which the projected data has maximal variance.

For centred data, the projection of $\tilde{\mathbf{x}}^{(i)}$ onto $\mathbf{u}$ is

$$\mathbf{u}^\top \tilde{\mathbf{x}}^{(i)}.$$

The empirical variance of these scalar projections is

$$\frac{1}{n}\sum_{i=1}^{n}\left(\mathbf{u}^\top \tilde{\mathbf{x}}^{(i)}\right)^2.$$

Definition 15.3.1 (First principal component)**.** The first principal component is the solution of

$$\max_{\mathbf{u}\in\mathbb{R}^d} \frac{1}{n}\sum_{i=1}^{n}\left(\mathbf{u}^\top \tilde{\mathbf{x}}^{(i)}\right)^2 \quad \text{s.t.} \quad \|\mathbf{u}\|_2 = 1.$$

Covariance matrix form

Define the sample covariance matrix

$$\Sigma = \frac{1}{n}\sum_{i=1}^{n} \tilde{x}^{(i)}\tilde{\mathbf{x}}^{(i)\top}.$$

Then

$$\frac{1}{n}\sum_{i=1}^{n}\left(\mathbf{u}^\top \tilde{\mathbf{x}}^{(i)\top}\right)^2 = \mathbf{u}^\top \Sigma \mathbf{u}.$$

So the problem becomes

$$\max_{\|\mathbf{u}\|_2=1} \mathbf{u}^\top \Sigma \mathbf{u}.$$

15.4 Eigenvector Characterisation

The optimisation above is solved by an eigenvector of the covariance matrix.

Theorem 15.4.1. Let Σ be the sample covariance matrix of centred data. Then the first principal component is any unit eigenvector corresponding to the largest eigenvalue of Σ.

Proof. We solve

$$\max_{\mathbf{u}} \mathbf{u}^\top \Sigma \mathbf{u} \quad \text{s.t.} \quad \mathbf{u}^\top \mathbf{u} = 1.$$

Using a Lagrange multiplier λ, define

$$\mathcal{L}(\mathbf{u}, \lambda) = \mathbf{u}^\top \Sigma \mathbf{u} - \lambda(\mathbf{u}^\top \mathbf{u} - 1).$$

Differentiate with respect to $\mathbf{u}$:

$$\nabla_{\mathbf{u}} \mathcal{L} = 2\Sigma \mathbf{u} - 2\lambda \mathbf{u}.$$

Setting the gradient to zero gives

$$\Sigma \mathbf{u} = \lambda \mathbf{u}.$$

So any stationary point is an eigenvector of Σ, with objective value

$$\mathbf{u}^\top \Sigma \mathbf{u} = \mathbf{u}^\top(\lambda \mathbf{u}) = \lambda \|\mathbf{u}\|_2^2 = \lambda.$$

Therefore maximising the objective is equivalent to choosing the eigenvector with the largest eigenvalue.

Interpretation

The first principal component is the direction along which the data varies the most.

15.5 Subsequent Principal Components

Once the first principal component is chosen, the second should capture as much remaining variance as possible while being orthogonal to the first, and so on.

Definition 15.5.1 (First k principal components)**.** The first k principal components are orthonormal vectors $\mathbf{u}_1, \ldots, \mathbf{u}_k$ solving

$$\max_{\mathbf{u}_1,\ldots,\mathbf{u}_k} \sum_{i=1}^{k} \mathbf{u}_j^\top \Sigma \mathbf{u}_j \quad \text{s.t.} \quad \mathbf{u}_j^\top \mathbf{u}_\ell = \delta_{j\ell}.$$

Theorem 15.5.2. The first k principal components are the orthonormal eigenvectors of Σ corresponding to the largest k eigenvalues

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d \geq 0.$$

Interpretation

- $\mathbf{u}_1$ - direction of largest variance;
- $\mathbf{u}_2$ - largest remaining variance orthogonal to $\mathbf{u}_1$;
- ...
- $\mathbf{u}_k$ - largest remaining variance orthogonal to all previous components.

15.6 Projection onto the Principal Subspace

Let $U_k = \begin{bmatrix} \mathbf{u}_1 & \mathbf{u}_2 & \cdots & \mathbf{u}_k \end{bmatrix} \in \mathbb{R}^{d\times k}$. Then the low-dimensional representation of a centred point $\tilde{\mathbf{x}}$ is

$$\mathbf{z} = U_k^\top \tilde{\mathbf{x}} \in \mathbb{R}^k.$$

Definition 15.6.1 (PCA encoding)**.** The PCA encoding of $\mathbf{x}$ into k dimensions is

$$\mathbf{z} = U_k^\top (\mathbf{x} - \bar{\mathbf{x}}).$$

Thus PCA projects the point orthogonally onto the k-dimensional principal subspace.

15.7 Reconstruction Error View

Among all k-dimensional orthonormal subspaces, PCA minimises

$$\sum_{i=1}^{n} \|\tilde{\mathbf{x}}^{(i)} - U_k U_k^\top \tilde{\mathbf{x}}^{(i)}\|_2^2.$$

This is the total squared reconstruction error after projection.

Theorem 15.7.1. The k-dimensional principal subspace spanned by the top k eigenvectors of Σ minimises the total reconstruction error.

Proof sketch. For any orthonormal U_k,

$$\|\tilde{\mathbf{x}}^{(i)}\|_2^2 = \|U_k U_k^\top \tilde{\mathbf{x}}^{(i)}\|_2^2 + \|\tilde{\mathbf{x}}^{(i)} - U_k U_k^\top \tilde{\mathbf{x}}^{(i)}\|_2^2$$

by orthogonal decomposition. Summing over i,

$$\sum_{i=1}^{n} \|\tilde{\mathbf{x}}^{(i)} - U_k U_k^\top \tilde{\mathbf{x}}^{(i)}\|_2^2 = \sum_{i=1}^{n} \|\tilde{\mathbf{x}}^{(i)}\|_2^2 - \sum_{i=1}^{n} \|U_k U_k^\top \tilde{\mathbf{x}}^{(i)}\|_2^2.$$

Since the first term is constant, minimising reconstruction error is equivalent to maximising the captured projected variance. Therefore the optimal U_k is given by the top k eigenvectors of Σ.

Takeaway

PCA maximises retained variance and equivalently minimises orthogonal reconstruction error.

15.8 Explained Variance

If the eigenvalues of Σ are

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d,$$

then λ_j is the variance explained by principal component $\mathbf{u}_j$.

Definition 15.8.1 (Explained variance ratio)**.** The proportion of total variance explained by the first k principal components is

$$\frac{\lambda_1 + \cdots + \lambda_k}{\lambda_1 + \cdots + \lambda_d}$$

This is often used to choose k:

- pick the smallest k such that explained variance exceeds, say, 90%, 95%, or 99%;
- or inspect the scree plot for an elbow.

15.9 Matrix Formulation

Let the centred data matrix be

$$X = \begin{bmatrix} \tilde{\mathbf{x}}^{(1)\top} \\ \tilde{\mathbf{x}}^{(2)\top} \\ \vdots \\ \tilde{\mathbf{x}}^{(n)\top} \end{bmatrix} \in \mathbb{R}^{n \times d}.$$

Then the sample covariance matrix is

$$\Sigma = \frac{1}{n} X^\top X.$$

So the PCA can be computed from the eigendecomposition of $X^\top X$.

Interpretation

PCA is fundamentally a spectral method: it extracts the dominant eigenspaces of the covariance structure.

Connection to SVD

In practice, PCA is often computed by singular value decomposition (SVD) rather than by explicitly forming Σ. Let

$$X = USV^\top$$

be the singular value decomposition of the centred data matrix X, where $U \in \mathbb{R}^{n \times n}$, $S \in \mathbb{R}^{n \times d}$, and $V \in \mathbb{R}^{d \times d}$. Then

$$X^\top X = VS^\top SV^\top.$$

So the columns of V are eigenvectors of $X^\top X$, hence principal directions.

Proposition 15.9.1. The right singular vectors of the centred data matrix are the principal directions of PCA.

Thus, PCA can be implemented stably and efficiently using SVD. This is especially useful when:

- d is large,
- n is large,
- or numerical stability matters.

PCA as Low-Rank Approximation

Another important interpretation is that PCA gives the best rank-k linear approximation of the data matrix. Let X_k denote the rank-k approximation obtained by truncating the SVD of X.

Theorem 15.9.2 (Eckart-Young, informal form)**.** Among all matrices M of rank at most k,

$$\|X - M\|_F^2$$

is minimised by the truncated SVD X_k.

PCA is the optimal low-rank approximation in Frobenius norm. This makes PCA central not only in statistics, but also in numerical linear algebra and compression.

15.10 Geometric Interpretation

Geometrically, PCA finds the best-fitting k-dimensional affine subspace for the data in the least-squares sense.

- For $k = 1$, PCA finds the best-fitting line through the mean.
- For $k = 2$, PCA finds the best-fitting plane through the mean.

More generally, PCA finds the orthogonal subspace that best captures the cloud of data points.

15.11 Whitening

Sometimes after PCA we rescale each principal coordinate by the inverse square root of its variance.

If the PCA coordinates are $\mathbf{z} = U^\top(\mathbf{x} - \bar{\mathbf{x}})$, and the corresponding variances are $\lambda_1, \ldots, \lambda_k$, then a whitened representation is

$$\mathbf{z}_{\text{white}} = \operatorname{diag}\left(\frac{1}{\sqrt{\lambda_1}}, \ldots, \frac{1}{\sqrt{\lambda_k}}\right)\mathbf{z}.$$

Whitening removes second-order correlations and normalises variances. This is useful in some preprocessing pipelines, though it may amplify noise in small-variance directions.

15.12 Practical Issues

Feature scaling

If features are on very different scales, PCA may be dominated by large-scale coordinates. In such cases, we often standardise each feature before PCA.

Mean centring is essential

Without centring, PCA does not behave as intended.

Sensitivity to outliers

Because PCA is based on squared error and covariance, it can be strongly affected by outliers.

Linear method only

PCA only captures linear subspace structure. Nonlinear manifolds require nonlinear methods such as kernel PCA, t-SNE, or UMAP.

15.13 Strengths and Limitations

Strengths

- Simple and mathematically elegant;
- computationally efficient;
- gives optimal linear low-rank approximation;
- useful for compression, denoising, visualisation, and preprocessing;
- interpretable via variance and orthogonal directions.

Limitations

- Only captures linear structure;
- sensitive to scaling and outliers;
- principal components maximise variance, not necessarily downstream predictive utility;
- components may be hard to interpret semantically.

References

[1] A. Ng and T. Ma, "Cs229: Machine learning - lecture notes," 2023.

[1]